TRRAIL and Error: Evaluating Multi-Agent NeSy LLMs for Logic, Law, and Medical Question Answering

Jonathan W. Morris* JOWEMORR@UCSC.EDU
Khin Saw Yone* KYONE@UCSC.EDU
Leilani H. Gilpin LGILPIN@UCSC.EDU
University of California, Santa Cruz, 1156 High Street, Santa Cruz, CA 95064

Abstract

Logical reasoning remains challenging for LLMs, particularly in domains where intermediate reasoning must be inspected and checked. Neuro-Symbolic (NeSy) methods such as SymbCoT, LogicLM, and ProSLM expose structured IRs, but many use static pipelines and are evaluated mainly on logic benchmarks. We investigate multi-agent self-correction in NeSy question answering across law, medicine, and logical reasoning, using Chain-of-Thought (CoT) as a baseline and gpt-5.4-nano and gpt-5.3-chat-latest as backbones. We also introduce the **TRRAIL** score, an LLM-as-judge rubric for observable reasoning artifacts. TRRAIL abbreviates **T**ransparent, **R**easoning-relevant, **R**easoning-grounded, **A**nswer-supported, **I**nterpretable, and **L**ogical. Across the tested configurations, refinement benefits the Prolog pipelines, whereas Lean refinement decreases several task and TRRAIL metrics. The results therefore support refinement as a configuration-dependent mechanism rather than a universal improvement. Backbone capability and judge choice also materially affect measured performance and TRRAIL scores, underscoring that artifact verifiability, predictive accuracy, and human trust are distinct questions.

1. Introduction

Large Language Models (LLMs) have proven remarkably capable at general Question Answering (QA). They are now standard tools for machine translation, QA, code generation, and many other domains. Despite these strengths, LLMs can be unreliable due to *hallucinations*. Combined with the black-box nature of neural networks, this complicates their use in safety-critical tasks, where mistakes can be costly. Many approaches seek to make LLMs and neural networks more interpretable, including post-hoc methods such as Shapley-value attribution (Lundberg & Lee, 2017) and LIME (Ribeiro et al., 2016), and mechanistic methods such as induction heads (Olsson et al., 2022) and circuit analysis (Wang et al., 2023). These methods typically explain features, tokens, or model components rather than provide a coarse-grained account of an end-to-end reasoning pipeline. Neuro-Symbolic (NeSy) AI instead combines neural language processing with structured symbolic artifacts that can be executed or checked, although those artifacts do not directly reveal the model's hidden computation. Recent NeSy LLMs include Logic-LM (Pan et al., 2023) and

*Denotes equal contributions.

ProSLM (Vakharia et al., 2024), which use solvers such as Z3 (de Moura & Bjørner, 2008), Prover9 (McCune, 2009), Prolog (Wielemaker et al., 2012), and Lean (Moura & Ullrich, 2021) to process intermediate representations (IRs).

However, many of these systems operate as single-pass pipelines, which we hypothesized was limiting the quality of the IR generated by the LLMs. To address this, we propose an agentic NeSy LLM framework that combines the natural language understanding of LLMs with the verifiability of symbolic solvers through a structured, observable pipeline. Our agent orchestration is built using LangGraph (LangChain, 2024), where specialized agent interactions are organized as a predefined workflow represented by nodes, which are the agents, and edges, which are strict communication paths between agents.

We evaluate our framework on reasoning-intensive QA tasks spanning legal, biomedical, and logic domains, comparing against Chain-of-Thought (CoT) prompting (Wei et al., 2022) as our baseline. We test two variations of our agentic NeSy framework, one using Prolog as the IR while the other uses Lean 4. We also compare against SymbCoT (Xu et al., 2024), and various other NeSy LLMs that use common symbolic solvers as IR. Beyond standard performance metrics, we introduce the **TRRAIL** score, short for *Transparent Reasoning-relevant Reasoning-grounded Answer-supported Interpretable Logical score*. TRRAIL uses an LLM-as-judge rubric to measure the formal-artifact verifiability and judge-rated readability of observable reasoning traces, not human trust or causal faithfulness. We test gpt-5.4-nano (OpenAI, 2026c) and gpt-5.3-chat-latest (OpenAI, 2026b) as backbones and judges and analyze how both choices affect the reported outcomes.

The contributions of this paper are as follows:

1. We propose a multi-agent QA framework that translates natural-language inputs into symbolic IRs that can be executed or checked by external solvers.
2. We propose the **TRRAIL score**, an LLM-as-judge rubric for evaluating formal-artifact verifiability and the readability of observable reasoning traces alongside task accuracy.
3. We introduce a dynamic self-correction mechanism in which agents communicate to resolve logical inconsistencies and expose a checkable reasoning trace. Refinement improves the tested Prolog configurations, but it is not universally beneficial: several Lean metrics decrease with refinement.
4. We evaluate 16 strategies across law, medicine, and logic benchmarks, showing that backbone and judge choice affect both task performance and the operational TRRAIL measurements.

2. Related Work

A number of NeSy LLMs have been created and tested, including LogicLM (Pan et al., 2023), LINC (Olausson et al., 2023), SymbCoT (Xu et al., 2024), LeanReasoner (Jiang et al., 2024), and FaithfulCoT (Lyu et al., 2023). These systems share a pattern: they represent LLM QA as symbolic problems and derive results through logical deduction or solvers such as constraint-satisfaction or Boolean-satisfiability engines. Some systems, such as LogicLM and FaithfulCoT, use different solvers depending on the dataset. Others, such as SymbCoT, represent symbolic problems using only LLMs by having an LLM mimic a proof assistant, freeing the system from a particular solver's

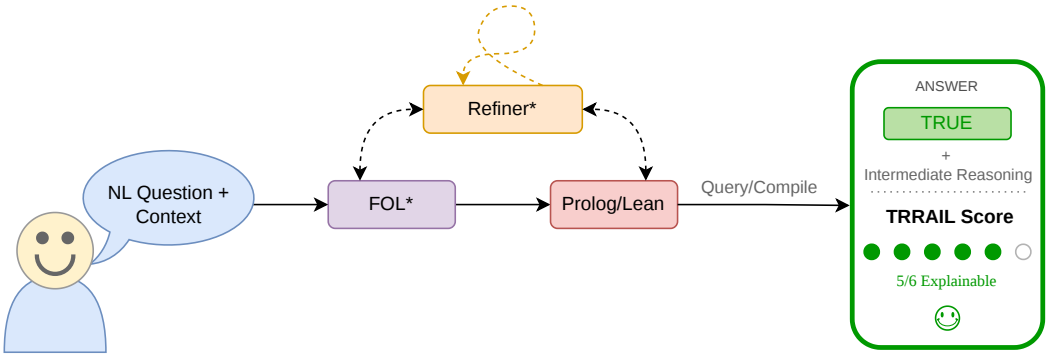


Figure 1. Condensed representation of Multi-agent (MA) NeSy LLM QA system that outputs an answer and IR, which is evaluated to produce a TRRAIL score. An asterisk denotes variable components of the system architecture.

syntax constraints. An LLM-only solver permits a single framework to span problem types, but its symbolic steps are not independently checked by an external formal engine.

The symbolic reasoners and representations considered include Prolog, Z3, Pyke (Frederiksen, 2008), Python, Prover9, Datalog (Gilray et al., 2024), Planning Domain Definition Language (PDDL) (Ghallab et al., 1998), and Lean 4. In this work we use Prolog and Lean to evaluate our agentic NeSy LLM framework and compare it against CoT prompting. We also include non-agentic NeSy results to compare the observed strengths and weaknesses of different symbolic tools used with LLM-generated IRs.

Our comparison between CoT and NeSy LLMs also reflects a broader distinction in how intelligent systems represent and organize reasoning. CoT prompting expresses intermediate reasoning as a sequence of natural-language statements, allowing an LLM to construct and communicate a reasoning process in the same representational space as the original problem. In contrast, NeSy approaches introduce an intermediate symbolic solver, providing a more structured mechanism for checking whether the intermediate reasoning is logically valid.

This is related to a broader body of research on cognitive architectures, which studies intelligent systems as organized collections of mechanisms for representing knowledge, reasoning, learning, memory, and decision making. Cognitive architectures such as ACT-R (Ritter et al., 2019) seek to provide a computational account of the information-processing mechanisms that support intelligent behavior. More generally, research on cognitive architectures emphasizes that intelligent behavior emerges from the organization and interaction of multiple system-level components, rather than from a single reasoning mechanism (Langley et al., 2009). This perspective motivates viewing LLM reasoning systems not only in terms of the quality of their final predictions, but also in terms of the representations, modules, and interactions through which those predictions are produced.

Our work follows this systems-level perspective by treating symbolic representations and formal solvers as components within an agentic reasoning architecture. Rather than relying exclusively on a natural-language chain of thought, our framework introduces structured IR that can be translated,

refined, and executed by symbolic reasoning tools. This provides an explicit interface between neural language understanding and formal reasoning, allowing the intermediate reasoning process to be inspected and evaluated independently of the final answer.

3. Datasets

We focus on LegalBench (Guha et al., 2023) and PubMedQA (Jin et al., 2019) for legal and biomedical question answering. Previous NeSy LLMs primarily use logic datasets such as FOLIO (Han et al., 2024) and ProofWriter (Tafjord et al., 2021); we therefore add domain-relevant benchmark tasks in law and medicine. We sampled 200 examples for each test from each dataset using a preset seed. The breakdown of the data set is as shown in table 5 in B.1.

FOLIO (Han et al., 2024) is a first-order logic reasoning benchmark dataset containing natural language premises, corresponding logical representations, and a hypothesis labeled as true, false, or uncertain. The dataset focuses on deductive reasoning, requiring systems to determine whether a conclusion logically follows from a set of premises.

PubMedQA (Jin et al., 2019) consists of biomedical research questions derived from scientific literature. Each instance contains a research question, an abstract from a biomedical publication, and a yes/no/maybe answer supported by the study findings.

LegalBench (Guha et al., 2023) is a benchmark derived from legal documents, contracts, policies, and terms-of-service agreements. The dataset evaluates the ability of models to interpret legal language and answer questions from a provided legal document. In this paper, we specifically use the consumer_contracts_qa data to test our models.

3.1 Data Pre-Processing

To enable consistent processing across these heterogeneous datasets, all examples were converted into a unified QA format consisting of three fields: context, question, and answer. For FOLIO, the original list of premises was concatenated into a single context passage, while the conclusion was reformulated as a natural-language question asking whether the statement could be inferred from the provided premises. The original labels (True, False, and Uncertain) were preserved as the ground truth. PubMedQA already closely matched the desired format and therefore required only minimal pre-processing. For LegalBench, contract clauses and policy excerpts were used as the context. This pre-processing procedure standardized the input structure across all datasets while preserving their underlying reasoning challenges. The resulting benchmark suite captures a diverse range of inference types, including formal logical deduction, scientific evidence-based reasoning, and legal text interpretation. Examples are in Appendix B.2.

4. Methodology

Our approach is a modular multi-agent symbolic reasoning framework that converts natural language reasoning tasks into executable formal representations Figure 1. We investigate multiple structured LangGraph (LangChain, 2024) architectures in which each reasoning stage is represented as a graph node with predefined communication paths. This structure enables controlled information

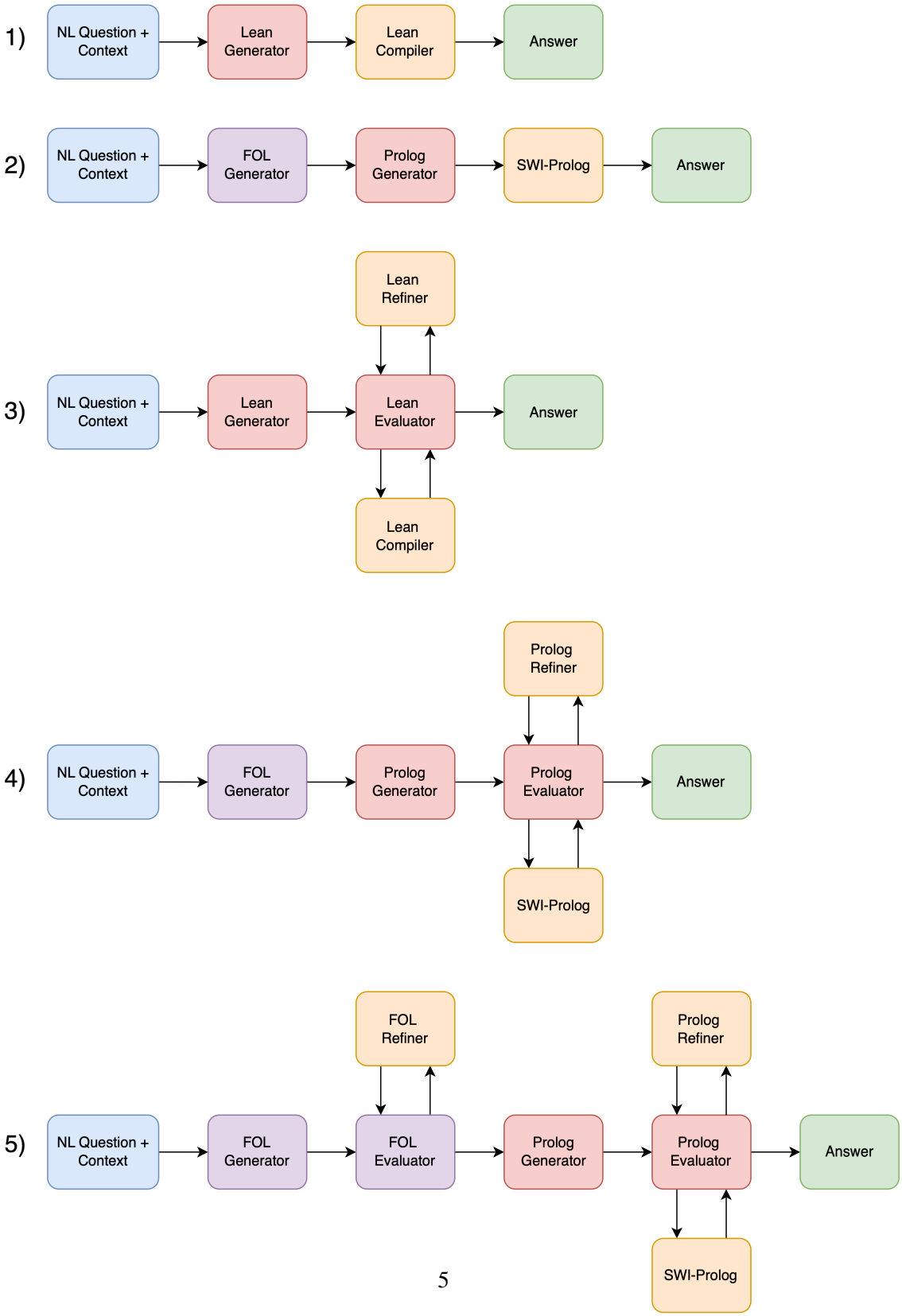


Figure 2. Overview of the five multi-agent pipeline variants evaluated in this work: (1) IR, (2) FOL-IR, (3) IR-Refiner, (4) FOL-IR-Refiner, and (5) Double-Refiner. The variants are described in detail in section 4.2.

flow and systematic ablation across different pipeline configurations. Our approach draws inspiration from prior work on faithful intermediate-representation-based reasoning, including (Xu et al., 2024) and (Lyu et al., 2023).

4.1 Modules

The framework consists of three primary module types: the Generator, Evaluator, and Refiner. Each module uses few-shot prompting to define the expected behavior and output format. The complete prompts are provided in Appendix C.

4.1.1 Generator

The Generator translates either natural language input or FOL into a target IR. The input consists of a question and its associated context. In this work, we consider two main IR: Prolog and Lean4.

Separate Generator modules are used for each target representation. The FOL Generator receives a natural language question and context and produces a Horn-style first-order logic representation. The Prolog Generator can operate directly on natural language or on a previously generated FOL representation and produces a SWI-Prolog knowledge base and query. Similarly, the Lean Generator produces Lean4 code from either natural language input or a previously generated FOL representation.

4.1.2 Evaluator

The Evaluator assesses whether a generated representation is sufficiently correct and faithful to proceed through the pipeline. Depending on the target representation, the Evaluator considers both the generated artifact and, when applicable, execution or compilation results.

The FOL Evaluator compares the generated FOL representation against the original natural language question and context and determines whether it is sufficiently faithful to pass to the next generation stage. The Prolog Evaluator examines the generated SWI-Prolog knowledge base and query together with the corresponding runtime results, evaluating both logical fidelity and executability. The Lean Evaluator operates similarly, using the generated Lean4 artifact together with its execution or compilation results. Each Evaluator returns either `APPROVE` or `ERROR`, with feedback provided in the latter case.

4.1.3 Refiner

The Refiner corrects defects identified by the Evaluator. It receives the original question and context, the current formal representation, and the Evaluator feedback, and returns a revised version of the representation.

The FOL Refiner edits the current FOL representation to improve its consistency with the original natural language input and the specified translation guidelines. The Prolog Refiner similarly revises the generated Prolog representation to ensure that it is executable and logically faithful to either the FOL representation or the original natural language input. Its output consists of a revised Prolog knowledge base and query. The Lean Refiner performs the corresponding revision process for Lean4 code.

4.2 Variants

We evaluate five main variants of the multi-agent (MA) QA system: *IR*, *fol-IR*, *IR-refiner*, *fol-IR-refiner*, and *double-refiner*. Figure 1 shows a condensed representation of the framework, while Figure 2 illustrates the variant configurations defined below.

1. *IR* configuration directly generates the target intermediate representation, either Prolog or Lean, from the natural language question and context. It uses only a Generator and does not perform evaluation or refinement.

2. *fol-IR* configuration introduces FOL as an intermediate reasoning stage. The system first generates a Horn-style FOL representation from the natural language input and then translates that representation into the target Prolog or Lean representation. No evaluation or refinement is performed at either stage.

3. *IR-refiner* configuration augments direct IR generation with an Evaluator–Refiner loop. After the initial intermediate representation is generated, the Evaluator either approves the representation or returns feedback to the Refiner. The Refiner then produces a corrected representation, which is passed back to the Evaluator.

4. *fol-IR-refiner* configuration first generates an FOL representation from the natural language input and then translates it into either Prolog or Lean. Refinement is applied only to the final Prolog or Lean representation, while the intermediate FOL representation is passed directly to the target-language Generator.

5. *double-refiner* configuration applies refinement at both stages. The system first generates and refines the FOL representation using an Evaluator–Refiner loop, then generates the target Prolog or Lean representation and applies a second Evaluator–Refiner loop.

For all configurations that use refinement, the Evaluator-Refiner loop continues until the representation is approved or a maximum of $N = 20$ refinement iterations is reached. We selected $N = 20$ based on preliminary experiments, in which $N = 10$ frequently terminated before satisfactory corrections were produced when using the smaller base model. The Evaluator prompts require the input context to be faithfully represented without shortcuts or logical fallacies; the full prompts are provided in Appendix C.

5. Evaluation

5.1 Performance Metrics

We report accuracy, precision, recall, F1 score, and error rate. Before evaluation, model outputs are normalized to the valid labels for each dataset; invalid outputs and execution failures are assigned to a dedicated *Error* class. Accuracy includes these errors as incorrect predictions. Precision, recall, and F1 are macro-averaged over the fixed set of dataset labels plus *Error*, with undefined classwise ratios set to zero (`zero_division=0`). Agentic models also return *Error* when the Evaluator does not approve the IR or FOL after $N = 20$ iterations. The error rate therefore captures failures to return an evaluable answer as well as invalid or failed executions.

5.2 Uncertainty Estimates

All subscripts and error bars are propagated bootstrap standard errors. For each dataset, we generated 1,000 paired record-level bootstrap samples with replacement from the 200 evaluation records, using seed 0, and recomputed each metric on every sample. The displayed point estimate is the metric on the original sample, and the dataset-level bootstrap standard error is the population standard deviation of its 1,000 bootstrap values. For a mean over the three datasets, the reported error is propagated as $\sqrt{\sum_d \mathrm{SE}_d^2}/3$. Thus, all bars denote ± 1 bootstrap standard error due to test-set composition; they are not confidence intervals, run-to-run variability, judge rerun variability, or significance tests. Where a figure further averages results across both backbones, its error bar is the arithmetic mean of component standard errors and is only a descriptive component-error summary, not a pooled standard error.

5.3 TRRAIL Score

Although benchmark performance is traditionally measured using task accuracy, accuracy alone does not describe whether an observable IR is checkable, logically consistent, or supportive of its conclusion. Models may achieve strong performance while exposing little evidence that can be independently inspected.

To address this limitation, we developed an LLM-judge-based measure, the **TRRAIL** score, defined in Table 1, that evaluates observable intermediate reasoning traces. The judge operates on the question, context, model response, predicted answer, ground truth, IR, and code-execution outputs when available. It assesses six artifact-oriented criteria derived from prior work on explainable AI, including Gilpin et al. (2018); it does not establish human trust, causal faithfulness, or deployment readiness. The criteria appear in Table 1, and the judge prompts appear in Appendix C.

Table 1. LLM-judge evaluation rubric, ordered as T, R_r, R_g, A, I, L. The formal-artifact-verifiability sub-score comprises T, R_g, and A; the judge-rated-readability sub-score comprises R_r, I, and L.

Metric	Description
Transparent, not merely persuasive	Does the explanation expose a checkable derivation rather than persuade through prose?
Reasoning relevant	Does the reasoning rely on information that meaningfully bears on the question?
Reasoning grounded	Is the reasoning expressed in a formal system that was actually executed or checked?
Answer supported	Does the final answer follow from a verified artifact rather than from prose assertion?
Interpretable reasoning	Can an expert examiner follow the explanation? (Symbolic artifacts count; fluency alone is not rewarded.)
Logical reasoning	Is the chain internally coherent and free of contradiction or circularity?

The judge answers "Yes" or "No" with a short justification for each criterion; the TRRAIL score is the number of "Yes" verdicts on a 0–6 scale. In acronym order, the criteria are **T**ransparent, **R**easoning relevant (R_r), **R**easoning grounded (R_g), **A**nswer supported, **I**nterpretable, and **L**ogical. We group R_r, I, and L as **judge-rated readability**, describing whether the trace is relevant, followable, and internally coherent. We group T, R_g, and A as **formal-artifact verifiability**, describing whether the trace exposes an executed or checked artifact that supports the answer. This second sub-score operationalizes what we call faithfulness in the figures; it does not identify the model's internal causal computation.

The judge receives an explanation of each criterion, few-shot calibration examples, an output-format specification, and configuration-specific input fields. For NeSy configurations it additionally sees the IR (e.g. FOL and solver code) and the standard output and error from executing it. We compare gpt-5.4-nano and gpt-5.3-chat-latest as judge models. After gpt-5.3-chat-latest was deprecated, the Prover9, Datalog, and Z3 configurations were judged with the rolling `chat-latest` alias (OpenAI, 2026a); those cells are not strictly controlled comparisons with the fixed gpt-5.3 judge runs.

TRRAIL is computed only for judge-eligible records. Records with an invalid answer, execution failure, evaluator rejection, or judge error are excluded from its denominator, so it characterizes accepted outputs rather than unconditional system behavior. Natural-language-only approaches such as CoT and SymbCoT do not pass their reasoning representations through an external formal solver and consequently receive zero or near-zero formal-artifact-verifiability scores. These scores do not show that their internal reasoning is inherently unfaithful; they show that the required formal evidence is absent under this rubric. We therefore report accuracy and error rate alongside TRRAIL and avoid treating TRRAIL as a substitute for predictive performance or human evaluation. In addition, we also provide judge reasoning along side each TRRAIL component for users to understand and reason with the judge's outputs.

6. Results

6.1 Task Performance

Table 2 summarizes task performance across FOLIO, LegalBench, and PubMedQA using the gpt-5.4-nano and gpt-5.3-chat-latest backbones. CoT obtains the highest displayed mean accuracy and F1 score. Moving to gpt-5.3-chat-latest raises mean accuracy across the 16 configurations by approximately 0.31 on FOLIO, 0.40 on LegalBench, and 0.31 on PubMedQA. Token reductions are concentrated in the refined FOL pipelines: mean token use falls from 27.6K to 6.0K for *ma-fol-lean-refiner* (about 78%) and from 28.3K to 10.9K for *ma-fol-prolog-refiner* (about 62%). The corresponding unrefined FOL pipelines do not halve token use, so token efficiency cannot be attributed to FOL pipelines generally.

The NeSy LLM systems generally achieve lower accuracy than CoT, although several configurations have similar macro precision. In particular, *ma-prolog-refiner* and *ma-fol-prolog-refiner* are the strongest multi-agent variants by mean accuracy: under gpt-5.3-chat-latest they reach 0.81 and 0.76, respectively. The zero-shot macro-precision estimates are backbone-dependent (0.442 for gpt-5.4-nano and 0.562 for gpt-5.3-chat-latest), so we do not use a common comparison thresh-

old. For many symbolic configurations, the larger precision–recall gap coincides with a high error rate, consistent with invalid, failed, or evaluator-rejected outputs contributing materially to lower accuracy.

The backbone-specific plots further illustrate this effect on symbolic pipelines. Figure 3 reports gpt-5.3-chat-latest, where precision, recall, and F1 remain relatively close for many configurations; CoT, *ma-prolog-refiner*, and *python* are particularly balanced. Figure 4 reports gpt-5.4-nano, where recall and F1 fall farther than precision for many symbolic configurations. This pattern co-occurs with higher error rates, indicating that invalid, failed, and evaluator-rejected outputs account for part of the performance loss; macro precision alone should not be read as conditional correctness on non-error records.

Performance also varies across datasets, but no dataset is uniformly hardest. Averaged across the 16 configurations, gpt-5.3-chat-latest attains mean accuracies of 0.691 on LegalBench, 0.644 on FOLIO, and 0.602 on PubMedQA. With gpt-5.4-nano, the order changes: FOLIO is highest at 0.335, followed by PubMedQA at 0.295 and LegalBench at 0.288. The data therefore support a backbone-dependent dataset pattern, not a general conclusion that one domain is intrinsically easiest or hardest.

6.2 Explainability and Reasoning Transparency

Across the reported judge/backbone conditions, matched Prolog configurations generally receive higher TRRAIL scores than their Lean counterparts (Table 4 and Figures 5–6). Executable NeSy configurations obtain high formal-artifact-verifiability scores while often remaining less accurate than CoT. This is an operational comparison under the rubric, not evidence that TRRAIL measures human transparency or trust.

Judge choice shifts score magnitude. Across the 32 reported configuration/backbone cells, the two judges have moderate rank agreement (Spearman $\rho \approx 0.60$), while gpt-5.3-chat-latest assigns scores about 0.99 points higher on average. The strongest multi-agent configurations, *ma-prolog-refiner* and *ma-fol-prolog-refiner*, keep mean macro precision within 3 percentage points of CoT under each backbone. The *ma-double-refiner-prolog* configuration receives a higher gpt-5.4-nano-judge TRRAIL score but lower precision, illustrating an observed tradeoff rather than a statistically tested system difference.

The criterion-level breakdown in Figures 5 and 6 shows that the two judges differ more in score magnitude than in the main architectural pattern. The gpt-5.3-chat-latest judge assigns near-ceiling scores to several executable symbolic systems, whereas gpt-5.4-nano produces greater separation among configurations. Across both evaluations, Prolog-based systems generally combine stronger faithfulness components with high readability, while Lean-based systems are more variable.

6.3 Readability versus Faithfulness Criterion

The distinction between judge-rated readability and formal-artifact verifiability is apparent when comparing CoT with the agentic NeSy LLMs: CoT scores zero on the verifiability sub-score while receiving near-ceiling readability scores. The proposed multi-agent architectures instead expose IRs, verification stages, and refinement procedures. These traces provide more explicit, checkable

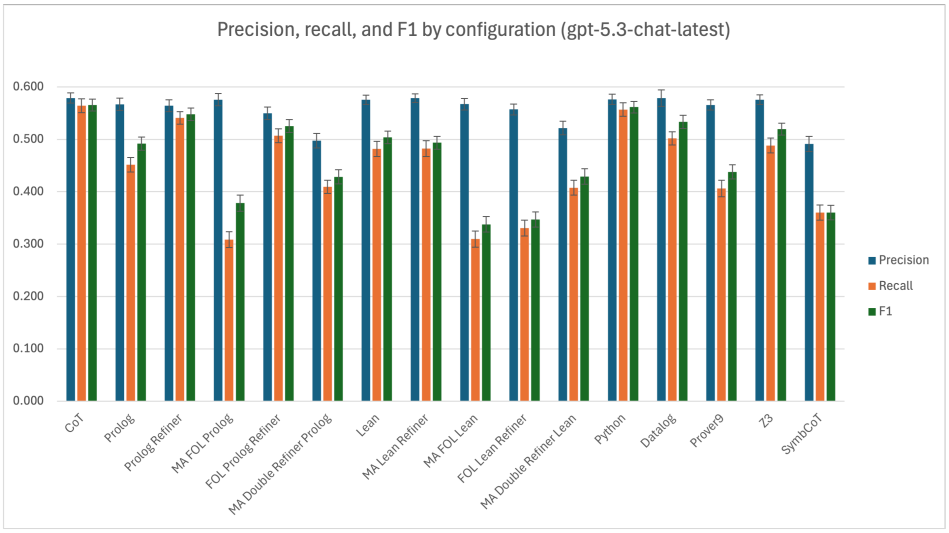


Figure 3. Mean precision, recall, and F1 score by reasoning configuration using the gpt-5.3-chat-latest backbone. Error bars show ± 1 propagated bootstrap standard error across datasets.

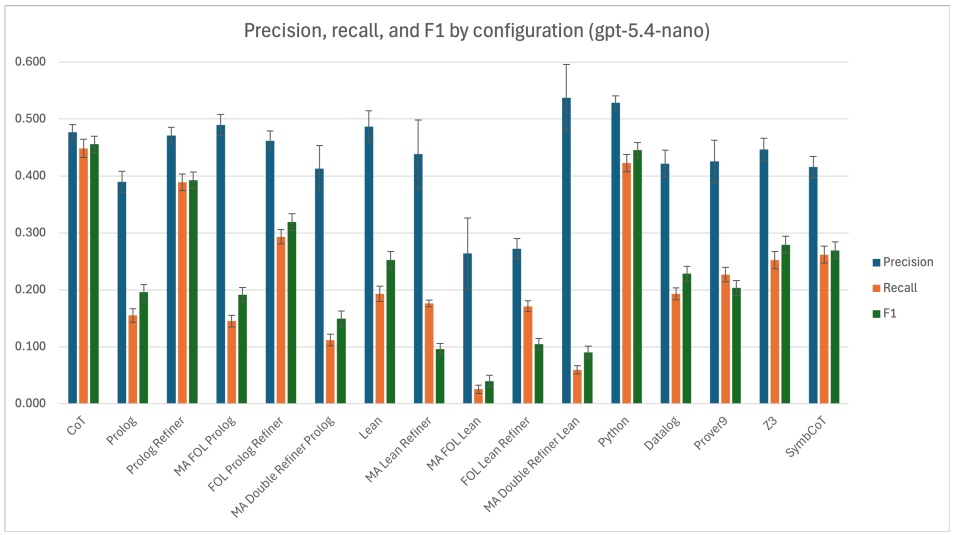


Figure 4. Mean precision, recall, and F1 score by reasoning configuration using the gpt-5.4-nano backbone. Error bars show ± 1 propagated bootstrap standard error across datasets.

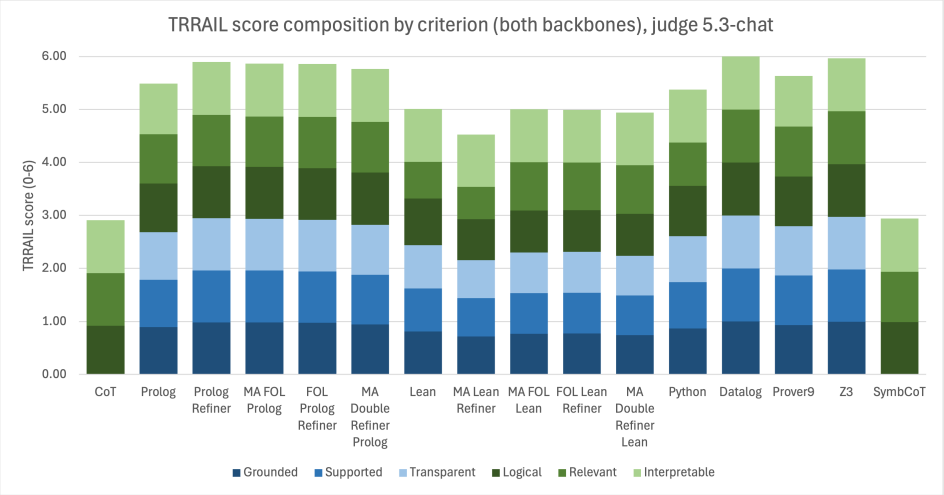


Figure 5. TRRAIL score composition by criterion and reasoning configuration, aggregated across both GPT backbones and evaluated by the gpt-5.3-chat-latest judge.

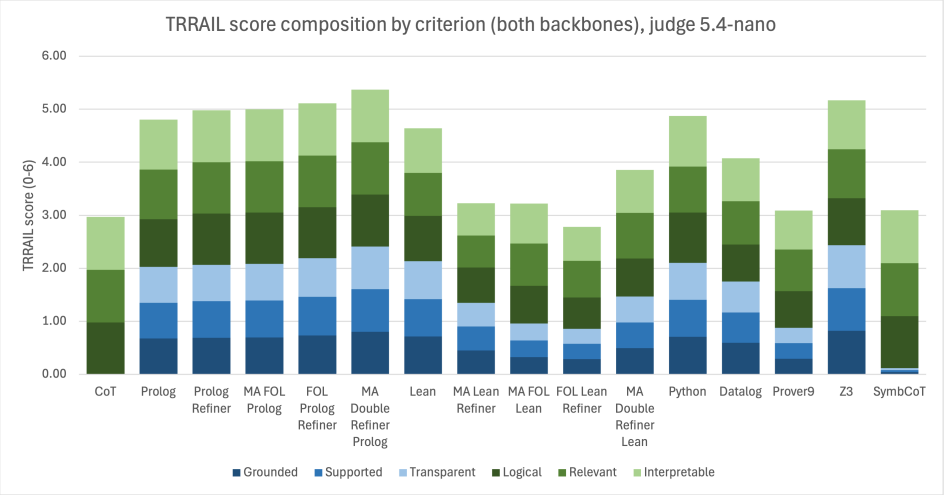


Figure 6. TRRAIL score composition by criterion and reasoning configuration, aggregated across both GPT backbones and evaluated by the gpt-5.4-nano judge.

Table 2. All 16 reasoning strategies on both backbones. Each cell is the mean over FOLIO, LegalBench, and PubMedQA with the propagated bootstrap standard error as a subscript ($\text{mean}_{\pm\text{SE}}$). **Acc**: accuracy, **Prec**: precision, **Expl**: TRRAIL score (0–6), **Err**: error rate, and **Tok**: mean tokens per sample (thousands). Standard errors use 1,000 paired record-level resamples (seed 0); they are not confidence intervals or run-to-run variability. Explainability scores are assigned by the gpt-5.4-nano judge. For each backbone, displayed maxima for Acc, Prec, and Expl and displayed minima for Err and Tok are in **bold**. The `ma-` prefix abbreviates `multi-agent-`.

	GPT-5.4-nano					GPT-5.3-chat-latest				
Configuration	Acc	Prec	Expl	Err	Tok	Acc	Prec	Expl	Err	Tok
`cot`	$\mathbf{.65}_{\pm.02}$	$.48_{\pm.01}$	$2.95_{\pm.01}$	$\mathbf{.04}_{\pm.01}$	$\mathbf{0.8}_{\pm0.0}$	$\mathbf{.82}_{\pm.02}$	$\mathbf{.58}_{\pm.01}$	$2.99_{\pm.01}$	$\mathbf{.01}_{\pm.00}$	$\mathbf{0.8}_{\pm0.0}$
`prolog`	$.24_{\pm.02}$	$.39_{\pm.02}$	$4.88_{\pm.10}$	$.58_{\pm.02}$	$1.9_{\pm0.0}$	$.67_{\pm.02}$	$.57_{\pm.01}$	$4.73_{\pm.07}$	$.18_{\pm.02}$	$2.0_{\pm0.0}$
`ma-prolog-refiner`	$.57_{\pm.02}$	$.47_{\pm.01}$	$5.11_{\pm.06}$	$.09_{\pm.01}$	$10.5_{\pm0.5}$	$.81_{\pm.02}$	$.56_{\pm.01}$	$4.84_{\pm.06}$	$.02_{\pm.01}$	$6.0_{\pm0.3}$
`ma-fol-prolog`	$.23_{\pm.02}$	$.49_{\pm.02}$	$5.08_{\pm.09}$	$.59_{\pm.02}$	$3.9_{\pm0.0}$	$.46_{\pm.02}$	$\mathbf{.58}_{\pm.01}$	$4.93_{\pm.08}$	$.40_{\pm.02}$	$3.9_{\pm0.0}$
`ma-fol-prolog-refiner`	$.45_{\pm.02}$	$.46_{\pm.02}$	$5.25_{\pm.07}$	$.28_{\pm.02}$	$28.3_{\pm1.2}$	$.76_{\pm.02}$	$.55_{\pm.01}$	$4.98_{\pm.06}$	$.06_{\pm.01}$	$10.9_{\pm0.6}$
`ma-double-refiner-prolog`	$.17_{\pm.02}$	$.41_{\pm.04}$	$\mathbf{5.62}_{\pm.07}$	$.71_{\pm.02}$	$41.1_{\pm0.8}$	$.63_{\pm.02}$	$.50_{\pm.01}$	$5.12_{\pm.06}$	$.10_{\pm.01}$	$14.1_{\pm0.6}$
`lean`	$.29_{\pm.02}$	$.49_{\pm.03}$	$4.81_{\pm.13}$	$.57_{\pm.02}$	$2.4_{\pm0.0}$	$.71_{\pm.02}$	$\mathbf{.58}_{\pm.01}$	$4.48_{\pm.10}$	$.08_{\pm.01}$	$2.5_{\pm0.0}$
`ma-lean-refiner`	$.16_{\pm.01}$	$.44_{\pm.06}$	$2.12_{\pm.10}$	$.36_{\pm.01}$	$15.5_{\pm0.5}$	$.67_{\pm.02}$	$\mathbf{.58}_{\pm.01}$	$4.34_{\pm.09}$	$.07_{\pm.01}$	$4.6_{\pm0.1}$
`ma-fol-lean`	$.03_{\pm.01}$	$.26_{\pm.06}$	$2.87_{\pm.30}$	$.95_{\pm.01}$	$5.1_{\pm0.0}$	$.43_{\pm.02}$	$.57_{\pm.01}$	$3.56_{\pm.10}$	$.29_{\pm.02}$	$5.6_{\pm0.0}$
`ma-fol-lean-refiner`	$.17_{\pm.01}$	$.27_{\pm.02}$	$2.13_{\pm.09}$	$.39_{\pm.01}$	$27.6_{\pm0.9}$	$.46_{\pm.02}$	$.56_{\pm.01}$	$3.43_{\pm.10}$	$.21_{\pm.01}$	$6.0_{\pm0.1}$
`ma-double-refiner-lean`	$.09_{\pm.01}$	$\mathbf{.54}_{\pm.06}$	$3.24_{\pm.18}$	$.83_{\pm.01}$	$60.0_{\pm0.9}$	$.61_{\pm.02}$	$.52_{\pm.01}$	$4.47_{\pm.09}$	$.13_{\pm.01}$	$14.6_{\pm0.5}$
`python`	$.61_{\pm.02}$	$.53_{\pm.01}$	$4.60_{\pm.08}$	$.11_{\pm.01}$	$1.5_{\pm0.0}$	$.81_{\pm.02}$	$\mathbf{.58}_{\pm.01}$	$5.14_{\pm.07}$	$.02_{\pm.01}$	$1.6_{\pm0.0}$
`datalog`	$.30_{\pm.02}$	$.42_{\pm.02}$	$4.18_{\pm.14}$	$.52_{\pm.02}$	$1.3_{\pm0.0}$	$.76_{\pm.02}$	$\mathbf{.58}_{\pm.02}$	$3.97_{\pm.10}$	$.12_{\pm.01}$	$1.5_{\pm0.0}$
`prover9`	$.26_{\pm.02}$	$.43_{\pm.04}$	$2.67_{\pm.09}$	$.35_{\pm.01}$	$1.4_{\pm0.0}$	$.55_{\pm.02}$	$.57_{\pm.01}$	$3.51_{\pm.08}$	$.19_{\pm.01}$	$1.6_{\pm0.0}$
`z3`	$.34_{\pm.02}$	$.45_{\pm.02}$	$4.85_{\pm.09}$	$.35_{\pm.02}$	$2.4_{\pm0.0}$	$.71_{\pm.02}$	$\mathbf{.58}_{\pm.01}$	$\mathbf{5.49}_{\pm.06}$	$.11_{\pm.01}$	$2.2_{\pm0.0}$
`symbcot`	$.34_{\pm.02}$	$.42_{\pm.02}$	$2.99_{\pm.01}$	$.26_{\pm.01}$	$9.0_{\pm0.1}$	$.46_{\pm.02}$	$.49_{\pm.01}$	$3.20_{\pm.03}$	$.19_{\pm.01}$	$6.8_{\pm0.0}$

artifacts under the rubric, although successful execution alone does not prove that the translation from natural language is correct or that the trace caused the model's answer.

These findings motivate reporting both sub-scores. **Judge-rated readability** estimates whether an expert examiner could follow the presented artifact. The verifiability sub-score records whether the answer is accompanied by executed or checked formal evidence. Neither sub-score was validated against human ratings, and neither identifies the model's internal causal reasoning process.

Figures 7 and 8 show this separation directly. CoT and SymbCoT lie near the high-readability, zero- or near-zero-verifiability boundary because they do not expose externally checked formal artifacts under TRRAIL. Other NeSy configurations occupy regions with nonzero scores on both dimensions. The gpt-5.3-chat-latest judge clusters many systems near the upper-right corner, whereas gpt-5.4-nano spreads them more widely and more clearly separates Prolog from Lean. This is consistent with different judge behavior around Lean artifacts, but the experiment does not isolate whether model capability, syntax familiarity, or another factor causes the difference.

7. Discussion

7.1 Performance Metrics

Dataset difficulty depends on the backbone. PubMedQA has the lowest mean accuracy with gpt-5.3-chat-latest, whereas LegalBench has the lowest mean with gpt-5.4-nano. We therefore cannot attribute a uniform difficulty ranking to domain alone. A notable configuration-level contrast is Python versus direct Prolog: Python has higher accuracy and lower error under both backbones (for example, 61.0% versus 23.8% accuracy and 11.3% versus 58.2% error with gpt-5.4-nano) at



Figure 7. Formal-artifact verifiability (labeled faithfulness in the plot) versus judge-rated readability, aggregated across both GPT backbones and evaluated by the gpt-5.3-chat-latest judge. Any displayed error summaries are descriptive averages of component standard errors, not pooled standard errors.

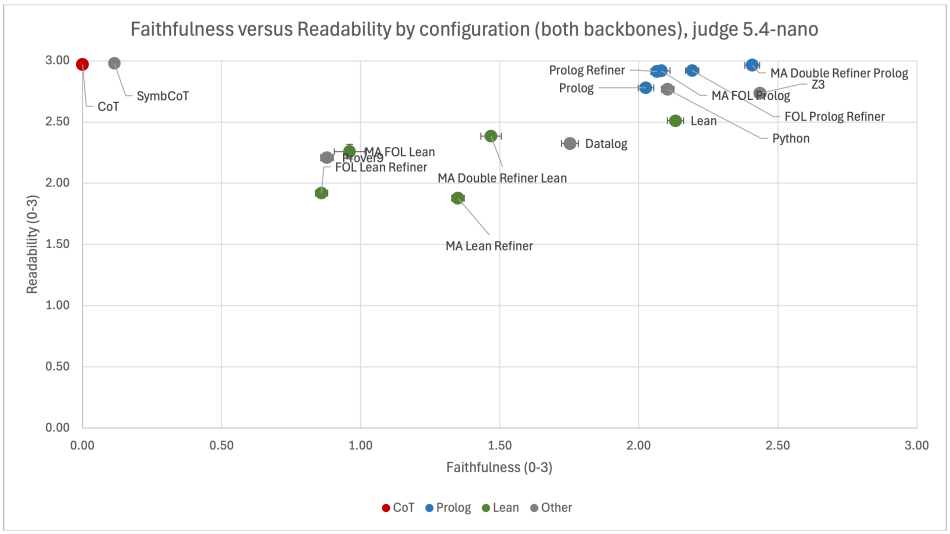


Figure 8. Formal-artifact verifiability (labeled faithfulness in the plot) versus judge-rated readability, aggregated across both GPT backbones and evaluated by the gpt-5.4-nano judge. Any displayed error summaries are descriptive averages of component standard errors, not pooled standard errors.

Table 3. Mean precision, recall, and F1 score for all 16 reasoning strategies across FOLIO, LegalBench, and PubMedQA. Propagated bootstrap standard errors appear as subscripts ($\mathrm{mean}_{\pm\mathrm{SE}}$), based on 1,000 paired record-level resamples (seed 0). Every displayed maximum in each metric and backbone is in **bold**.

	GPT-5.4-nano			GPT-5.3-chat-latest		
Configuration	Prec	Recall	F1	Prec	Recall	F1
cot	$.48_{\pm.01}$	$\mathbf{.45}_{\pm.02}$	$\mathbf{.46}_{\pm.01}$	$\mathbf{.58}_{\pm.01}$	$\mathbf{.56}_{\pm.01}$	$\mathbf{.57}_{\pm.01}$
prolog	$.39_{\pm.02}$	$.15_{\pm.01}$	$.20_{\pm.01}$	$.57_{\pm.01}$	$.45_{\pm.01}$	$.49_{\pm.01}$
ma-prolog-refiner	$.47_{\pm.01}$	$.39_{\pm.01}$	$.39_{\pm.01}$	$.56_{\pm.01}$	$.54_{\pm.01}$	$.55_{\pm.01}$
ma-fol-prolog	$.49_{\pm.02}$	$.15_{\pm.01}$	$.19_{\pm.01}$	$\mathbf{.58}_{\pm.01}$	$.31_{\pm.02}$	$.38_{\pm.02}$
ma-fol-prolog-refiner	$.46_{\pm.02}$	$.29_{\pm.01}$	$.32_{\pm.01}$	$.55_{\pm.01}$	$.51_{\pm.01}$	$.53_{\pm.01}$
ma-double-refiner-prolog	$.41_{\pm.04}$	$.11_{\pm.01}$	$.15_{\pm.01}$	$.50_{\pm.01}$	$.41_{\pm.01}$	$.43_{\pm.01}$
lean	$.49_{\pm.03}$	$.19_{\pm.01}$	$.25_{\pm.02}$	$\mathbf{.58}_{\pm.01}$	$.48_{\pm.01}$	$.50_{\pm.01}$
ma-lean-refiner	$.44_{\pm.06}$	$.18_{\pm.01}$	$.10_{\pm.01}$	$\mathbf{.58}_{\pm.01}$	$.48_{\pm.02}$	$.49_{\pm.01}$
ma-fol-lean	$.26_{\pm.06}$	$.03_{\pm.01}$	$.04_{\pm.01}$	$.57_{\pm.01}$	$.31_{\pm.02}$	$.34_{\pm.01}$
ma-fol-lean-refiner	$.27_{\pm.02}$	$.17_{\pm.01}$	$.10_{\pm.01}$	$.56_{\pm.01}$	$.33_{\pm.02}$	$.35_{\pm.01}$
ma-double-refiner-lean	$\mathbf{.54}_{\pm.06}$	$.06_{\pm.01}$	$.09_{\pm.01}$	$.52_{\pm.01}$	$.41_{\pm.01}$	$.43_{\pm.01}$
python	$.53_{\pm.01}$	$.42_{\pm.02}$	$.45_{\pm.01}$	$\mathbf{.58}_{\pm.01}$	$\mathbf{.56}_{\pm.01}$	$.56_{\pm.01}$
datalog	$.42_{\pm.02}$	$.19_{\pm.01}$	$.23_{\pm.01}$	$\mathbf{.58}_{\pm.02}$	$.50_{\pm.01}$	$.53_{\pm.01}$
prover9	$.43_{\pm.04}$	$.23_{\pm.01}$	$.20_{\pm.01}$	$.57_{\pm.01}$	$.41_{\pm.02}$	$.44_{\pm.01}$
z3	$.45_{\pm.02}$	$.25_{\pm.02}$	$.28_{\pm.01}$	$\mathbf{.58}_{\pm.01}$	$.49_{\pm.01}$	$.52_{\pm.01}$
symbcot	$.42_{\pm.02}$	$.26_{\pm.01}$	$.27_{\pm.01}$	$.49_{\pm.01}$	$.36_{\pm.01}$	$.36_{\pm.01}$

Table 4. Per-criterion TRRAIL pass rates (%) for all 16 reasoning strategies using the gpt-5.3-chat-latest backbone and judge, averaged across FOLIO, LegalBench, and PubMedQA. Propagated bootstrap standard errors appear as subscripts ($\mathrm{mean}_{\pm\mathrm{SE}}$), based on 1,000 paired record-level resamples (seed 0). Columns follow the TRRAIL acronym: **T**: transparent, R_r: reasoning relevant, R_g: reasoning grounded, **A**: answer supported, **I**: interpretable, and **L**: logical. Column maxima are in **bold**; ties at the displayed precision are all bolded.

	GPT-5.3-chat-latest Judge					
Configuration	T	R_r	R_g	A	I	L
cot	$0.00_{\pm0.00}$	$99.83_{\pm0.16}$	$0.00_{\pm0.00}$	$0.00_{\pm0.00}$	$\mathbf{100.00}_{\pm0.00}$	$98.17_{\pm0.55}$
prolog	$98.20_{\pm0.60}$	$96.99_{\pm0.75}$	$98.20_{\pm0.60}$	$98.20_{\pm0.60}$	$99.82_{\pm0.19}$	$98.21_{\pm0.60}$
ma-prolog-refiner	$99.83_{\pm0.16}$	$97.66_{\pm0.61}$	$99.83_{\pm0.16}$	$99.83_{\pm0.16}$	$\mathbf{100.00}_{\pm0.00}$	$99.83_{\pm0.16}$
ma-fol-prolog	$98.59_{\pm0.63}$	$98.70_{\pm0.57}$	$98.59_{\pm0.63}$	$98.59_{\pm0.63}$	$\mathbf{100.00}_{\pm0.00}$	$99.17_{\pm0.47}$
ma-fol-prolog-refiner	$99.67_{\pm0.23}$	$98.98_{\pm0.42}$	$99.67_{\pm0.23}$	$99.67_{\pm0.23}$	$\mathbf{100.00}_{\pm0.00}$	$99.26_{\pm0.37}$
ma-double-refiner-prolog	$96.24_{\pm0.78}$	$99.32_{\pm0.33}$	$96.24_{\pm0.78}$	$96.24_{\pm0.78}$	$\mathbf{100.00}_{\pm0.00}$	$99.45_{\pm0.31}$
lean	$79.47_{\pm1.28}$	$80.46_{\pm1.62}$	$79.47_{\pm1.28}$	$79.47_{\pm1.28}$	$\mathbf{100.00}_{\pm0.00}$	$85.83_{\pm1.20}$
ma-lean-refiner	$79.00_{\pm1.18}$	$81.33_{\pm1.58}$	$79.00_{\pm1.18}$	$79.00_{\pm1.18}$	$99.67_{\pm0.23}$	$83.33_{\pm1.17}$
ma-fol-lean	$78.27_{\pm1.36}$	$94.30_{\pm0.98}$	$78.27_{\pm1.36}$	$78.09_{\pm1.35}$	$99.81_{\pm0.19}$	$82.53_{\pm1.35}$
ma-fol-lean-refiner	$80.00_{\pm1.24}$	$96.17_{\pm0.78}$	$80.00_{\pm1.24}$	$80.00_{\pm1.24}$	$\mathbf{100.00}_{\pm0.00}$	$83.50_{\pm1.27}$
ma-double-refiner-lean	$80.83_{\pm1.50}$	$97.43_{\pm0.67}$	$80.83_{\pm1.50}$	$80.83_{\pm1.50}$	$99.81_{\pm0.19}$	$87.50_{\pm1.32}$
python	$92.33_{\pm1.06}$	$80.50_{\pm1.55}$	$92.33_{\pm1.06}$	$92.33_{\pm1.06}$	$\mathbf{100.00}_{\pm0.00}$	$98.67_{\pm0.47}$
datalog	$\mathbf{100.00}_{\pm0.00}$	$\mathbf{100.00}_{\pm0.00}$	$\mathbf{100.00}_{\pm0.00}$	$\mathbf{100.00}_{\pm0.00}$	$\mathbf{100.00}_{\pm0.00}$	$\mathbf{100.00}_{\pm0.00}$
prover9	$99.64_{\pm0.26}$	$\mathbf{100.00}_{\pm0.00}$	$99.64_{\pm0.26}$	$99.64_{\pm0.26}$	$\mathbf{100.00}_{\pm0.00}$	$99.64_{\pm0.26}$
z3	$99.82_{\pm0.18}$	$99.82_{\pm0.18}$	$99.82_{\pm0.18}$	$99.82_{\pm0.18}$	$\mathbf{100.00}_{\pm0.00}$	$99.82_{\pm0.18}$
symbcot	$0.33_{\pm0.23}$	$99.00_{\pm0.41}$	$0.33_{\pm0.23}$	$0.33_{\pm0.23}$	$\mathbf{100.00}_{\pm0.00}$	$99.00_{\pm0.41}$

comparable token cost. Python execution earns formal-artifact-verifiability credit under TRRAIL, but that credit does not imply that an arbitrary program constitutes a valid logical derivation. This comparison illustrates that accuracy, execution success, and the strength of formal guarantees are separate design considerations.

The results also separate **judge-rated readability** from **formal-artifact verifiability**. CoT and SymbCoT score highly on the former and zero or near zero on the latter because they do not expose an externally checked formal artifact. This operational distinction is useful for comparing what a system makes available for inspection, but it does not show that a natural-language trace is causally unfaithful or that a formal trace matches human judgments.

The choice of formal representation materially affects the observed outcomes. Matched Lean configurations generally have lower task and TRRAIL metrics than their Prolog counterparts, and Lean refinement does not reproduce the Prolog gains. Producing Lean artifacts entails satisfying syntax, typing, library, and proof-structure constraints, but this study does not isolate why the Lean and Prolog results differ. The evidence supports a configuration-specific difference in generation and evaluation success, not a claim about either model's training exposure.

Within the tested **Prolog** configurations, refinement improves accuracy and reduces error relative to the corresponding unrefined pipelines. Iterative feedback can expose syntax errors or incomplete rule sets for revision. The same pattern does not hold for Lean: refinement decreases several metrics. Adding a verifier or refinement loop therefore does not automatically compensate for a difficult IR, and the observed Prolog benefit should not be generalized to all formalisms.

7.2 TRRAIL Metrics

TRRAIL is judge-dependent. Across the 32 reported configuration/backbone cells, gpt-5.3-chat-latest assigns scores about 0.99 points higher on average than gpt-5.4-nano. Nevertheless, their rankings have moderate agreement (Spearman $\rho \approx 0.60$). The higher scores and near-ceiling clustering from gpt-5.3-chat-latest reduce discrimination among some systems, while gpt-5.4-nano spreads scores more widely. These are descriptive differences in judge output; without human ratings, they do not establish which judge is more accurate or which better represents expert or novice readers.

Formal checking and accessibility should also remain distinct. Lean supplies a structured proof environment, yet its artifacts receive lower judge-rated readability than Prolog in these experiments. A successful check establishes a property of the generated formal artifact, but not necessarily the correctness of the natural-language translation or its accessibility to a human reader. Evaluations of symbolic systems should therefore report execution, task correctness, and independently validated readability rather than treating any one as a proxy for the others.

The ablations suggest that, for Prolog, refinement contributes more to task performance than the intermediate FOL stage. Adding FOL without refinement does not improve accuracy over direct Prolog generation under either backbone, and FOL-plus-refinement remains less accurate than the simpler Prolog-refiner configuration. FOL can expose an additional translation stage, but it increases token cost and adds another potential failure point. Among the tested multi-agent configurations, the single Prolog refinement loop is therefore one of the strongest observed tradeoffs between accuracy, artifact verifiability, and cost; this is a descriptive multi-objective judgment, not an optimized or statistically tested ranking.

Finally, the Prover9, Datalog, and Z3 judge runs used the rolling `chat-latest` alias after gpt-5.3-chat-latest was deprecated. Because the alias can change over time, these cells should not be

treated as directly comparable to fixed-version gpt-5.3 judge runs. We retain them for completeness and identify the mixed judge provenance when interpreting cross-configuration patterns.

7.3 Limitations

Each system produced one inference output per sampled record, so the study does not measure run-to-run model variability. Each dataset contributes 200 examples, which limits coverage of its full task distribution and of the broader legal, biomedical, and logical-reasoning domains. The record-level bootstrap quantifies sensitivity to the sampled test records only. Its ± 1 standard-error summaries are not confidence intervals or tests of pairwise system differences; we make no inferential claim about differences between systems.

TRRAIL depends on an LLM judge, exhibits ceiling effects for several configurations, and has not been validated against human annotations. The judge receives the ground-truth answer, which may improve assessment of answer support but also makes the rubric unsuitable as a blind evaluation of the trace alone. TRRAIL is also conditional on judge-eligible outputs: error and judge-failure records are excluded, so high scores can coexist with low coverage. CoT and SymbCoT are structurally disadvantaged on the formal-artifact-verifiability sub-score because they do not expose externally checked artifacts. In addition, the replacement of the deprecated gpt-5.3 judge with `chat-latest` for Prover9, Datalog, and Z3 introduces uncontrolled model-version variation.

Finally, the three selected benchmarks and two model backbones do not establish generalization to other datasets, solvers, models, or real legal and clinical workflows. We did not evaluate domain-expert use, downstream decisions, safety outcomes, deployment readiness, or whether the formal artifacts are readable to people. These remain empirical questions for human-centered and prospective evaluation.

8. Future Work

Reducing refinement cost is a priority for NeSy LLMs. Several configurations with high formal-artifact-verifiability scores are also expensive; learning when to stop refining, or when refinement is unlikely to help, could preserve useful artifacts at lower cost. Fine-tuning or otherwise adapting models to formal languages may also improve translation from natural language to IR, but this requires direct evaluation.

Future work should improve and validate TRRAIL with human ratings, blinded inputs, and more discriminative judge prompts. Complementary sufficiency and comprehensiveness measures (DeYoung et al., 2020) could test whether removing information from an artifact changes the prediction. Intervention-based methods (Meng et al., 2022) could separately test causal relationships between internal or intermediate states and final answers. These additions would address questions that the present artifact-based rubric cannot answer.

9. Conclusion

We presented a multi-agent Neuro-Symbolic LLM question-answering framework that translates natural language into executable symbolic representations and can refine intermediate artifacts.

We also introduced TRRAIL, an LLM-as-judge rubric that separates judge-rated readability from formal-artifact verifiability. Across three 200-example benchmark samples, CoT has the highest displayed mean task accuracy, while executable NeSy systems expose artifacts that receive higher verifiability scores under the rubric.

Refinement is not a universal benefit. It improves the tested Prolog configurations, whereas Lean refinement decreases several metrics; the additional FOL stage also does not consistently improve task performance. Backbone choice strongly affects accuracy, error rate, and token use, while judge choice shifts TRRAIL magnitude and produces only moderate rank agreement. These results support reporting predictive performance, failures, artifact verifiability, judge-rated readability, and computational cost together. They do not establish human trust, causal faithfulness, or readiness for legal or medical deployment, but they identify concrete tradeoffs for future human-validated studies of checkable reasoning artifacts.

References

de Moura, L., & Bjørner, N. (2008). Z3: An efficient SMT solver. *2008 Tools and Algorithms for Construction and Analysis of Systems* (pp. 337–340). Springer, Berlin, Heidelberg.

DeYoung, J., Jain, S., Rajani, N. F., Lehman, E., Xiong, C., Socher, R., & Wallace, B. C. (2020). ERASER: A benchmark to evaluate rationalized NLP models. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 4443–4458). Online: Association for Computational Linguistics. From https://aclanthology.org/2020.acl-main.408/.

Frederiksen, B. (2008). Applying expert system technology to code reuse with pyke. *PyCon: Proceedings of the Python Conference*.

Ghallab, M., et al. (1998). *PDDL: The planning domain definition language*. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control.

Gilpin, L. H., Bau, D., Yuan, B. Z., Bajwa, A., Specter, M., & Kagal, L. (2018). Explaining explanations: An overview of interpretability of machine learning. *2018 IEEE 5th International Conference on Data Science and Advanced Analytics (DSAA)* (pp. 80–89).

Gilray, T., Sahebolamri, A., Sun, Y., Kunapaneni, S., Kumar, S., & Micinski, K. (2024). Datalog with First-Class Facts. *Proceedings of the VLDB Endowment*, *18*, 651–665. From https://dl.acm.org/doi/10.14778/3712221.3712232.

Guha, N., et al. (2023). Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models. *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. From https://openreview.net/forum?id=WqSPQFxFRC.

Han, S., et al. (2024). FOLIO: Natural language reasoning with first-order logic. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (pp. 22017–22031). Miami, Florida, USA: Association for Computational Linguistics. From https://aclanthology.org/2024.emnlp-main.1229/.

Jiang, D., Fonseca, M., & Cohen, S. B. (2024). LeanReasoner: Boosting complex logical reasoning with lean. *Proceedings of the 2024 Conference of the North American Chapter of the Association*

for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers) (pp. 7497–7510). Mexico City, Mexico: Association for Computational Linguistics. From https://aclanthology.org/2024.naacl-long.416/.

Jin, Q., Dhingra, B., Liu, Z., Cohen, W., & Lu, X. (2019). PubMedQA: A dataset for biomedical research question answering. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 2567–2577). Hong Kong, China: Association for Computational Linguistics. From https://aclanthology.org/D19-1259/.

LangChain (2024). Langgraph. From https://github.com/langchain-ai/langgraph. Accessed: 2026-06-15.

Langley, P., Laird, J. E., & Rogers, S. (2009). Cognitive architectures: Research issues and challenges. *Cognitive Systems Research*, *10*, 141–160. From https://www.sciencedirect.com/science/article/pii/S1389041708000557.

Lundberg, S., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. From http://arxiv.org/abs/1705.07874. ArXiv:1705.07874 [cs.AI].

Lyu, Q., Havaldar, S., Stein, A., Zhang, L., Rao, D., Wong, E., Apidianaki, M., & Callison-Burch, C. (2023). Faithful chain-of-thought reasoning. *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 305–329). Nusa Dua, Bali: Association for Computational Linguistics. From https://aclanthology.org/2023.ijcnlp-main.20/.

McCune, W. (2009). Prover9 and mace4. From http://www.cs.unm.edu/~mccune/prover9/.

Meng, K., Bau, D., Andonian, A., & Belinkov, Y. (2022). Locating and editing factual associations in gpt. *Proceedings of the 36th International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc.

Moura, L. d., & Ullrich, S. (2021). The lean 4 theorem prover and programming language. *Automated Deduction – CADE 28* (pp. 625–635). Cham: Springer International Publishing.

Olausson, T., Gu, A., Lipkin, B., Zhang, C., Solar-Lezama, A., Tenenbaum, J., & Levy, R. (2023). LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 5153–5176). Singapore: Association for Computational Linguistics. From https://aclanthology.org/2023.emnlp-main.313/.

Olsson, C., et al. (2022). In-context Learning and Induction Heads. From http://arxiv.org/abs/2209.11895. ArXiv:2209.11895 [cs.LG].

OpenAI (2026a). Chat Latest model documentation. From https://developers.openai.com/api/docs/models/chat-latest. Accessed: 2026-08-24.

OpenAI (2026b). GPT-5.3 Chat model documentation. From https://developers.openai.com/api/docs/models/gpt-5.3-chat-latest. Accessed: 2026-08-24.

OpenAI (2026c). GPT-5.4 nano model documentation. From https://developers.openai.com/api/docs/models/gpt-5.4-nano. Accessed: 2026-08-24.

Pan, L., Albalak, A., Wang, X., & Wang, W. (2023). Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. *Findings of the Association for Computational Linguistics: EMNLP 2023* (pp. 3806–3824). Singapore: Association for Computational Linguistics. From https://aclanthology.org/2023.findings-emnlp.248/.

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1135–1144). New York, NY, USA: Association for Computing Machinery. From https://dl.acm.org/doi/10.1145/2939672.2939778.

Ritter, F. E., Tehranchi, F., & Oury, J. D. (2019). Act-r: A cognitive architecture for modeling cognition. *WIREs Cognitive Science*, *10*, e1488. From https://wires.onlinelibrary.wiley.com/doi/abs/10.1002/wcs.1488.

Tafjord, O., Dalvi, B., & Clark, P. (2021). ProofWriter: Generating implications, proofs, and abductive statements over natural language. *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021* (pp. 3621–3634). Online: Association for Computational Linguistics. From https://aclanthology.org/2021.findings-acl.317/.

Vakharia, P., Kufeldt, A., Meyers, M., Lane, I., & Gilpin, L. H. (2024). *Proslm: A prolog synergized language model for explainable domain specific knowledge based question answering*, (p. 291–304). Springer Nature Switzerland. From http://dx.doi.org/10.1007/978-3-031-71170-1_23.

Wang, K. R., Variengien, A., Conmy, A., Shlegeris, B., & Steinhardt, J. (2023). Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 Small. *International Conference on Learning Representations*. From https://openreview.net/forum?id=NpsVSN6o4ul.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., brian ichter, Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain of thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*. From https://openreview.net/forum?id=_VjQlMeSB_J.

Wielemaker, J., Schrijvers, T., Triska, M., & Lager, T. (2012). SWI-Prolog. *Theory and Practice of Logic Programming*, *12*, 67–96.

Xu, J., Fei, H., Pan, L., Liu, Q., Lee, M.-L., & Hsu, W. (2024). Faithful logical reasoning via symbolic chain-of-thought. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 13326–13365). Bangkok, Thailand: Association for Computational Linguistics. From https://aclanthology.org/2024.acl-long.720/.

Appendix A. Supplementary Figures

Unless a caption states otherwise, error bars in single-backbone panels show ± 1 propagated bootstrap standard error across datasets (1,000 paired record-level resamples per dataset, seed 0). In panels aggregated across both backbones, the plotted error summary is the arithmetic mean of the component standard errors; it is descriptive and is not a pooled standard error or a confidence interval. In plot labels and captions, "faithfulness" denotes only the TRRAIL formal-artifact-verifiability sub-score (T, R_g, A), not causal faithfulness or human trust.

A.1 Task Performance and Computational Efficiency

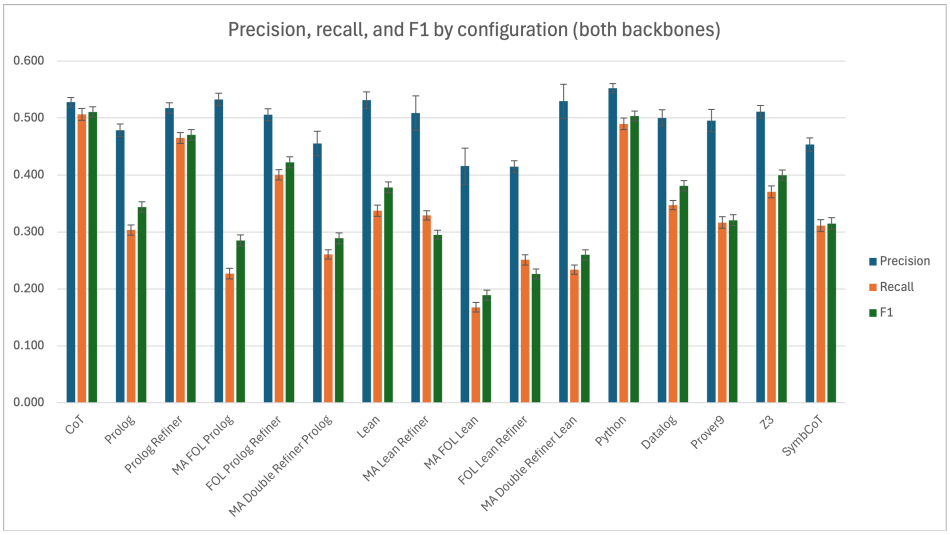


Figure 9. Mean precision, recall, and F1 score by reasoning configuration, aggregated across the gpt-5.3-chat-latest and gpt-5.4-nano backbones.

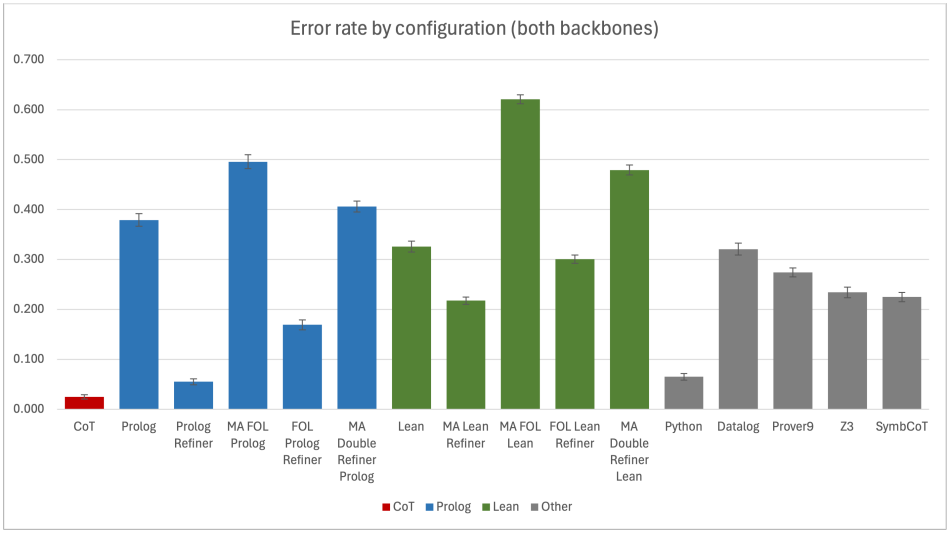


Figure 10. Mean error rate by reasoning configuration, aggregated across the gpt-5.3-chat-latest and gpt-5.4-nano backbones.

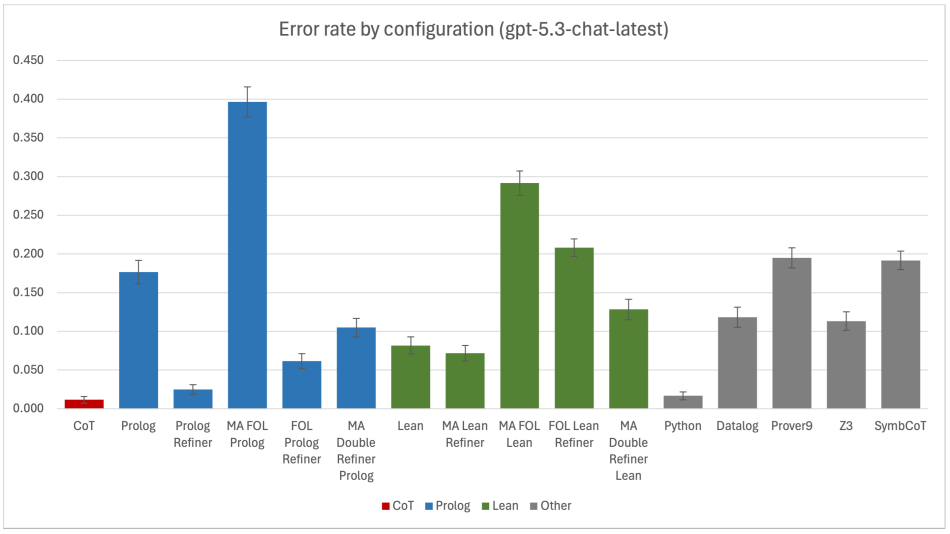


Figure 11. Mean error rate by reasoning configuration using the gpt-5.3-chat-latest backbone.

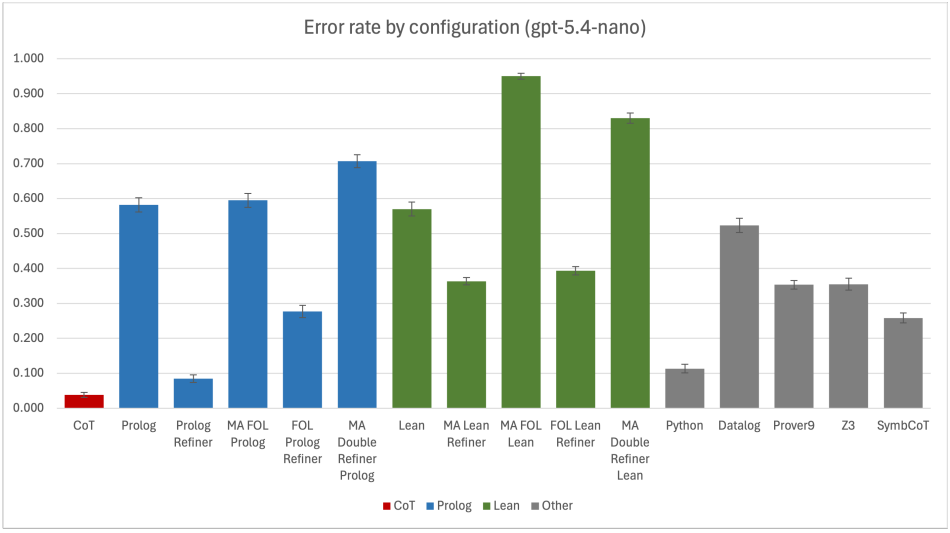


Figure 12. Mean error rate by reasoning configuration using the gpt-5.4-nano backbone.

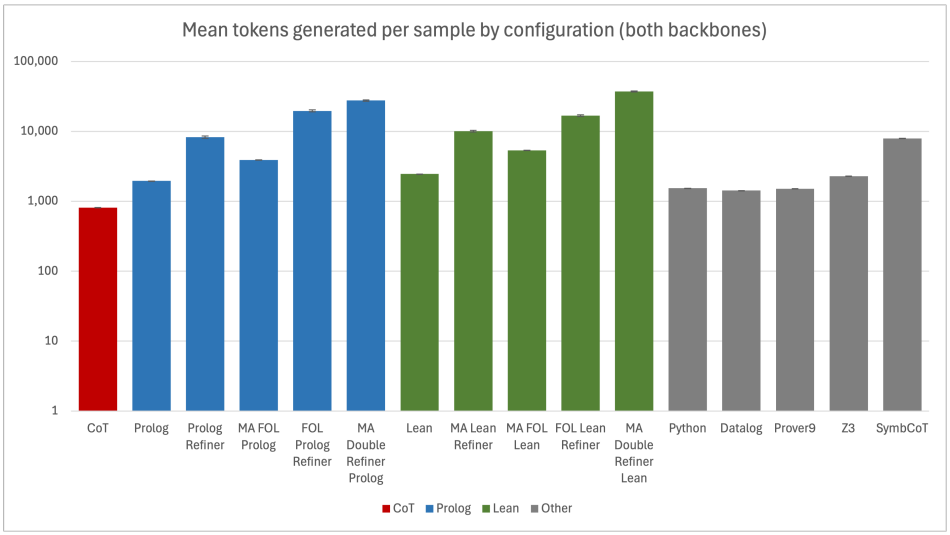


Figure 13. Mean token use by reasoning configuration, aggregated across the gpt-5.3-chat-latest and gpt-5.4-nano backbones.

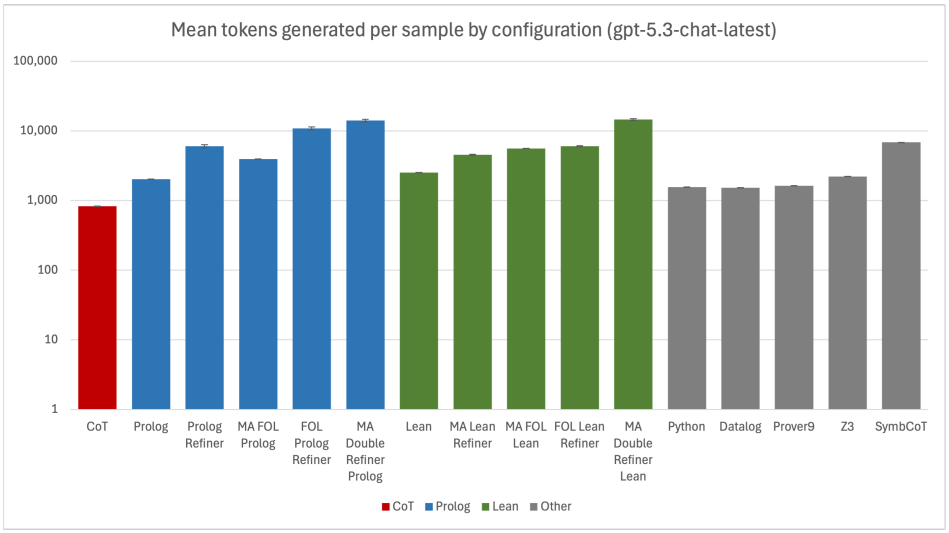


Figure 14. Mean token use by reasoning configuration using the gpt-5.3-chat-latest backbone.

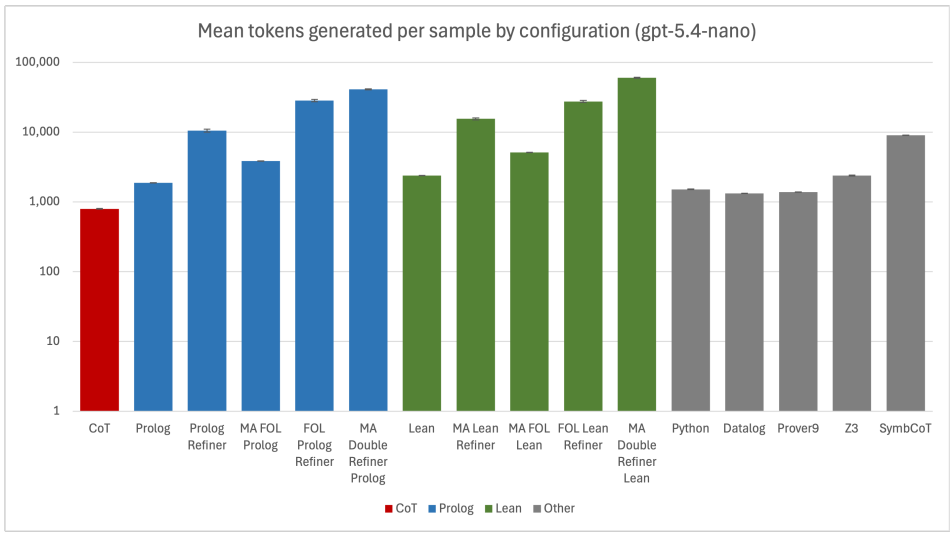


Figure 15. Mean token use by reasoning configuration using the gpt-5.4-nano backbone.

A.2 Accuracy–TRRAIL Relationships

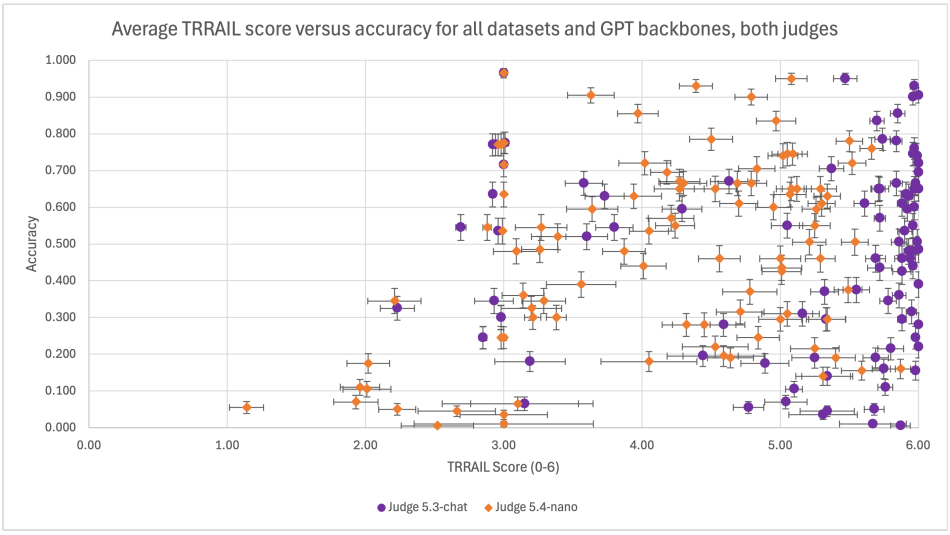


Figure 16. Average TRRAIL score versus accuracy for all datasets and both GPT backbones, comparing scores assigned by the gpt-5.3-chat-latest and gpt-5.4-nano judges.

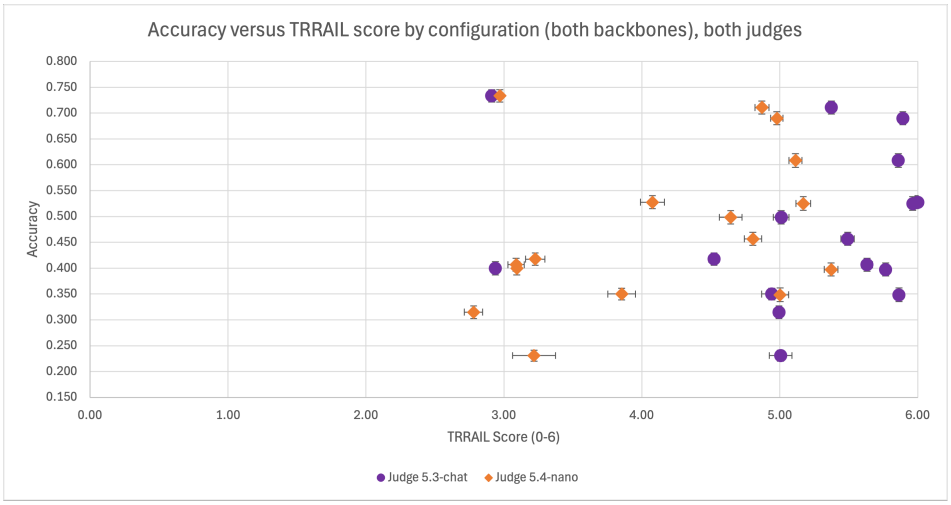


Figure 17. Accuracy versus mean TRRAIL score by reasoning configuration, aggregated across both GPT backbones and shown for both judge models.

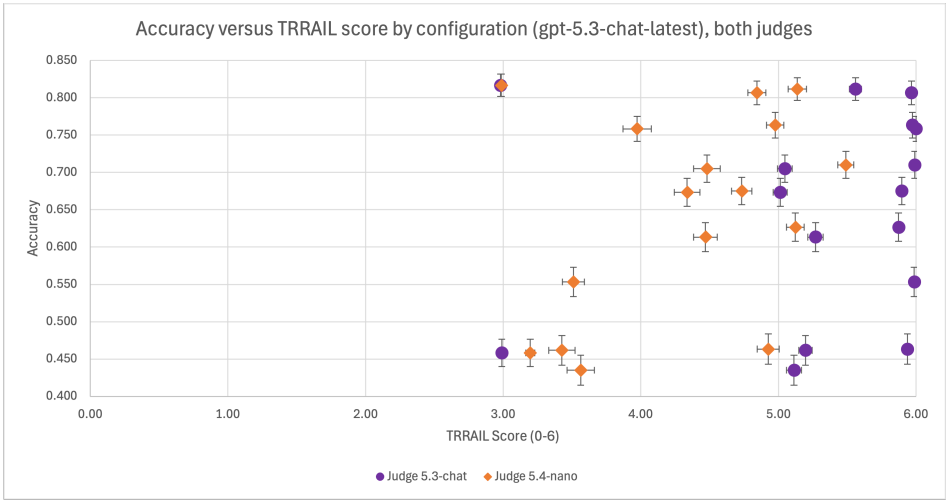


Figure 18. Accuracy versus mean TRRAIL score by reasoning configuration using the gpt-5.3-chat-latest backbone, shown for both judge models.

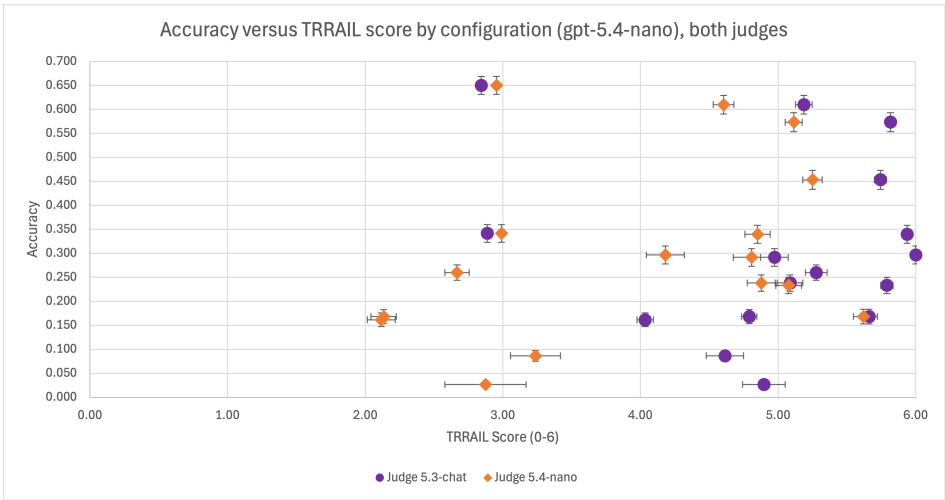


Figure 19. Accuracy versus mean TRRAIL score by reasoning configuration using the gpt-5.4-nano backbone, shown for both judge models.

A.3 Aggregate TRRAIL Scores

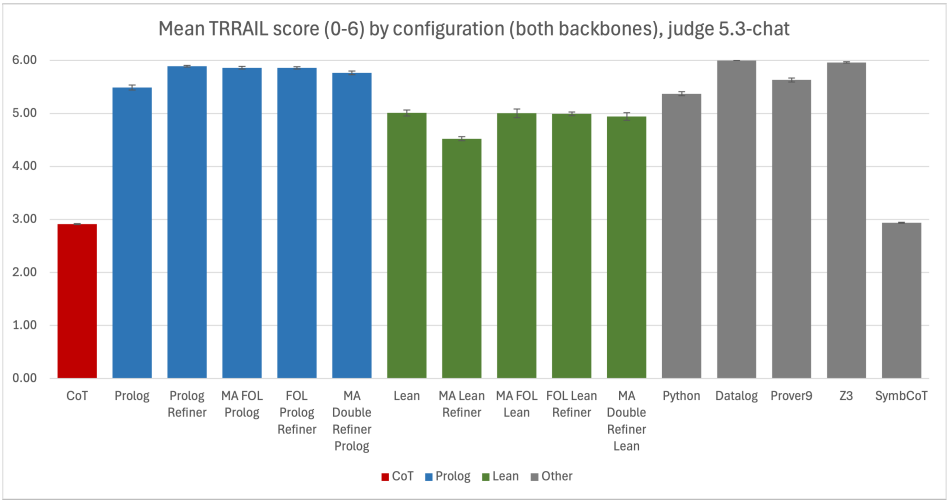


Figure 20. Mean TRRAIL score by reasoning configuration, aggregated across both GPT backbones and evaluated by the gpt-5.3-chat-latest judge.

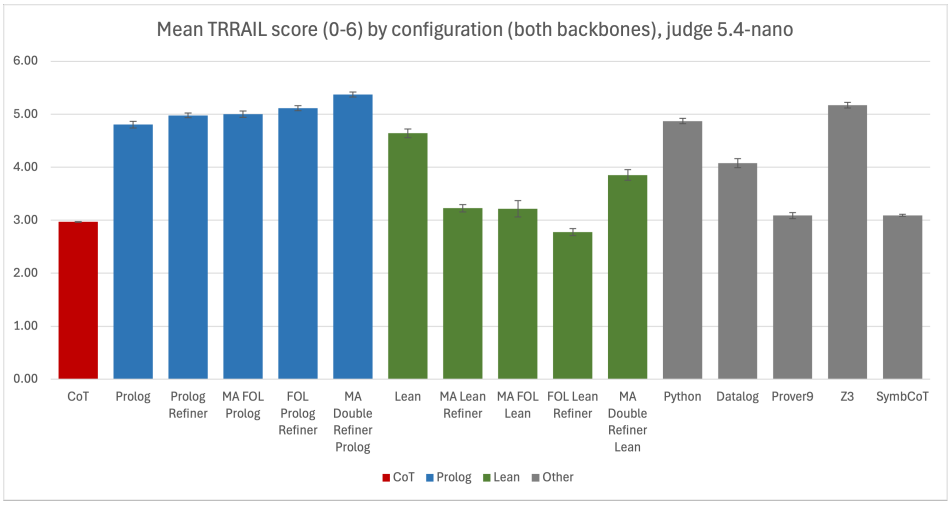


Figure 21. Mean TRRAIL score by reasoning configuration, aggregated across both GPT backbones and evaluated by the gpt-5.4-nano judge.

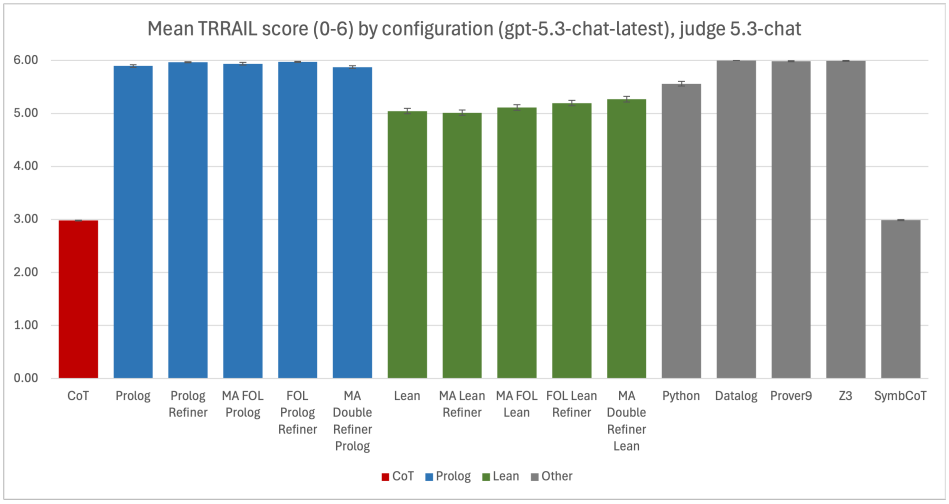


Figure 22. Mean TRRAIL score by reasoning configuration using the gpt-5.3-chat-latest backbone and the gpt-5.3-chat-latest judge.

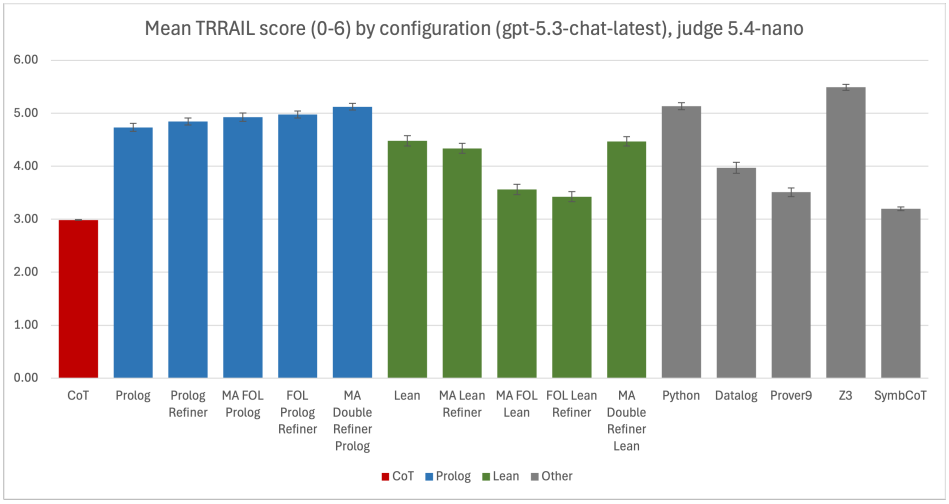


Figure 23. Mean TRRAIL score by reasoning configuration using the gpt-5.3-chat-latest backbone and the gpt-5.4-nano judge.

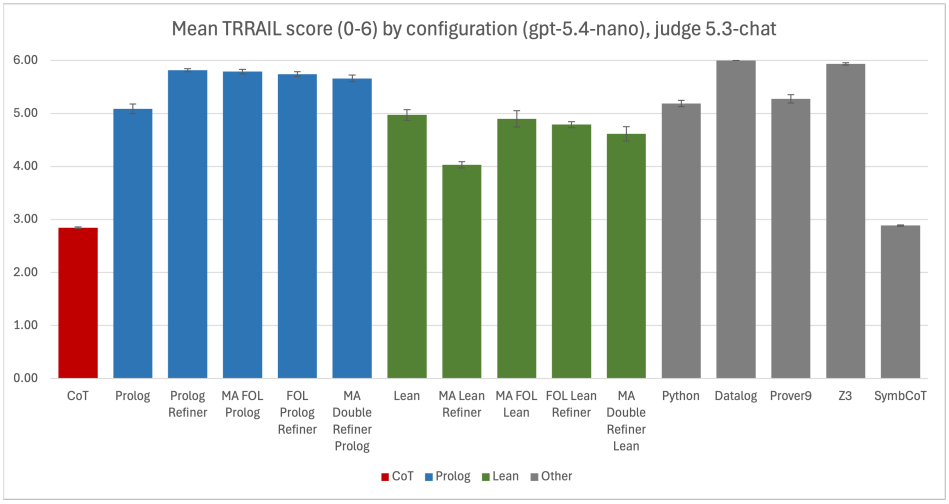


Figure 24. Mean TRRAIL score by reasoning configuration using the gpt-5.4-nano backbone and the gpt-5.3-chat-latest judge.

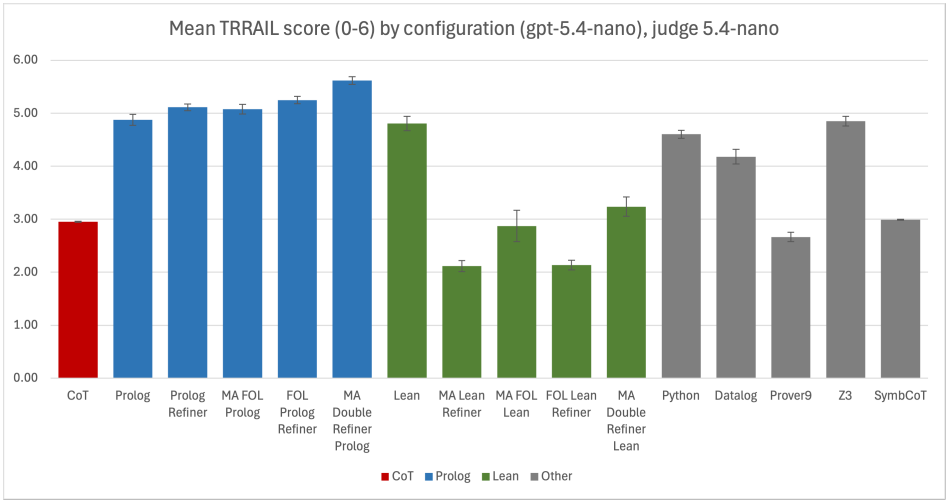


Figure 25. Mean TRRAIL score by reasoning configuration using the gpt-5.4-nano backbone and the gpt-5.4-nano judge.

A.4 TRRAIL Component Scores

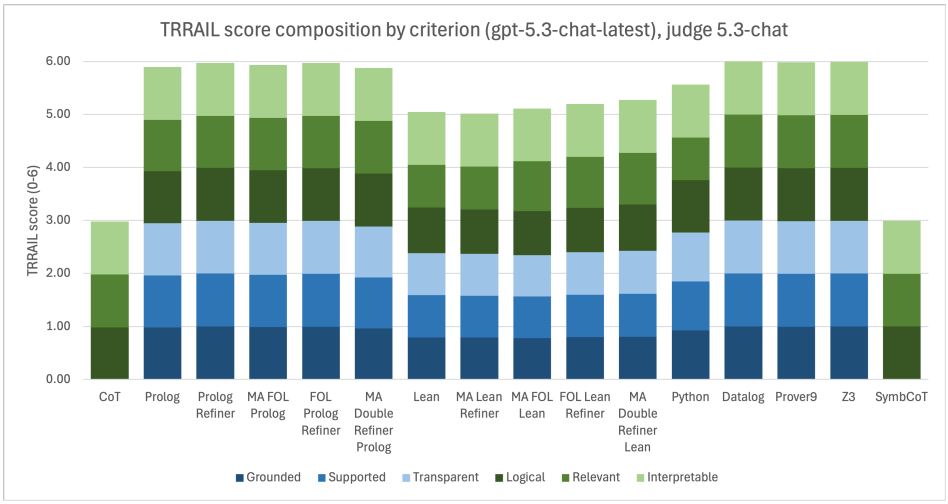


Figure 26. TRRAIL score composition by criterion and reasoning configuration using the gpt-5.3-chat-latest backbone and the gpt-5.3-chat-latest judge.

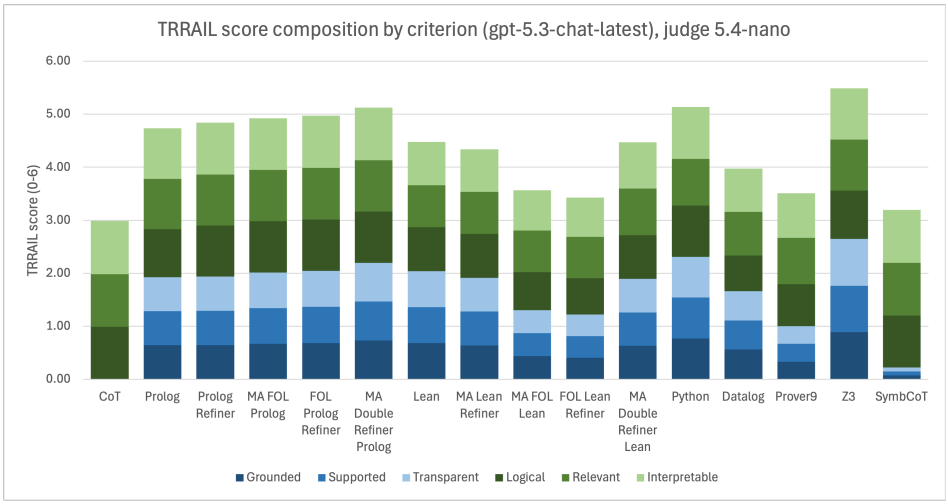


Figure 27. TRRAIL score composition by criterion and reasoning configuration using the gpt-5.3-chat-latest backbone and the gpt-5.4-nano judge.

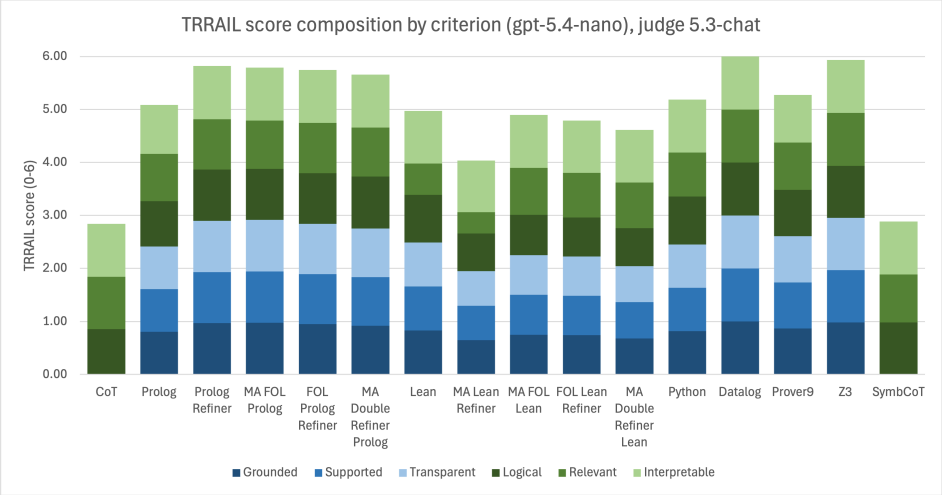


Figure 28. TRRAIL score composition by criterion and reasoning configuration using the gpt-5.4-nano backbone and the gpt-5.3-chat-latest judge.

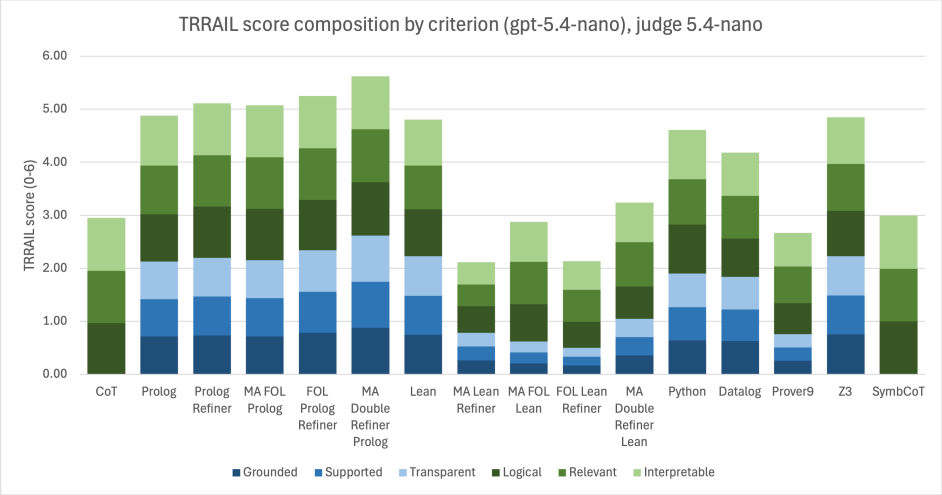


Figure 29. TRRAIL score composition by criterion and reasoning configuration using the gpt-5.4-nano backbone and the gpt-5.4-nano judge.

A.5 Faithfulness and Readability Sub-Scores

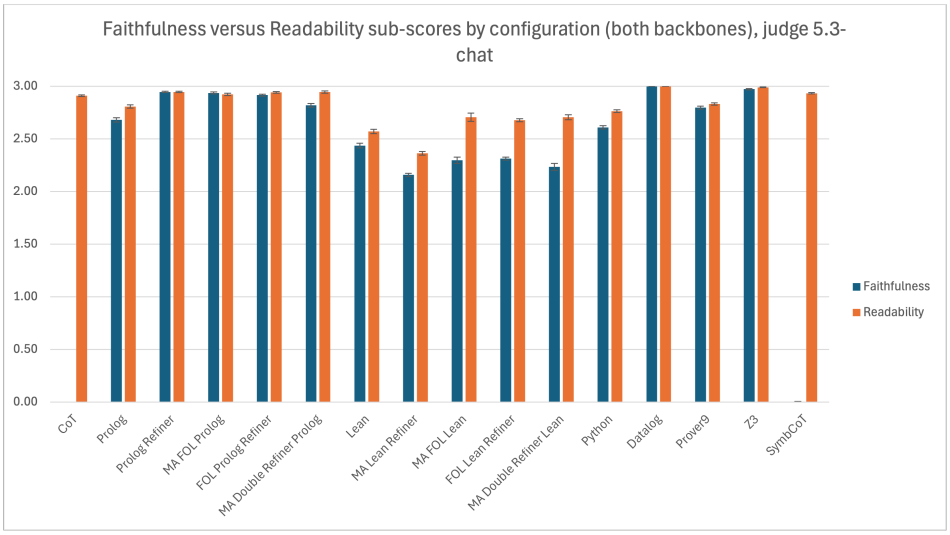


Figure 30. Mean faithfulness and readability sub-scores by reasoning configuration, aggregated across both GPT backbones and evaluated by the gpt-5.3-chat-latest judge.

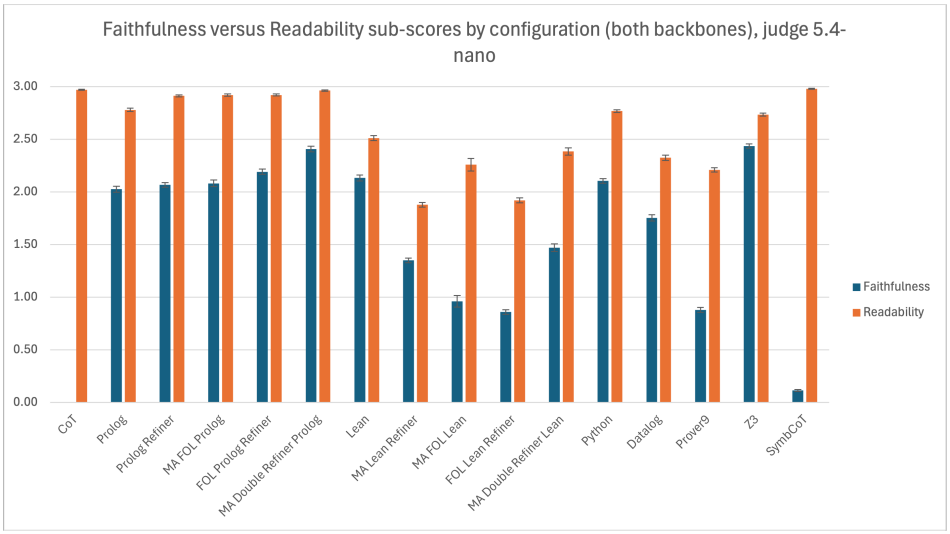


Figure 31. Mean faithfulness and readability sub-scores by reasoning configuration, aggregated across both GPT backbones and evaluated by the gpt-5.4-nano judge.

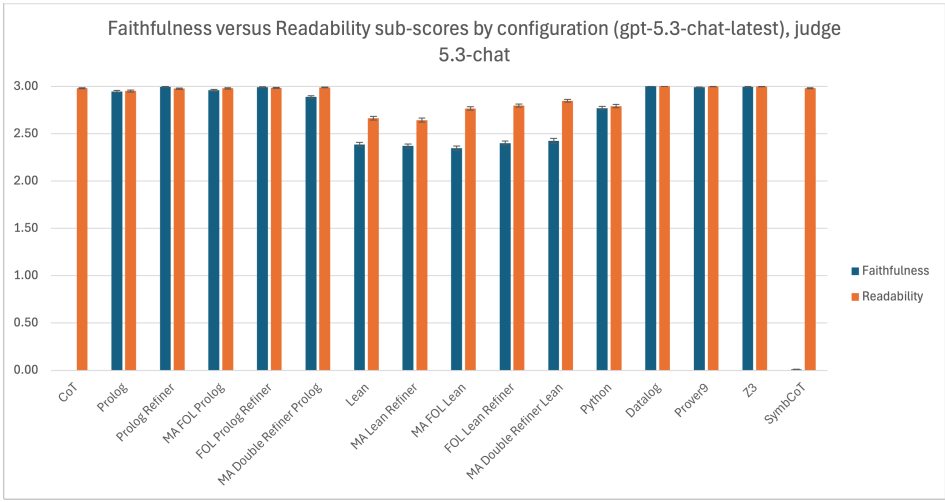


Figure 32. Mean faithfulness and readability sub-scores by reasoning configuration using the gpt-5.3-chat-latest backbone and the gpt-5.3-chat-latest judge.

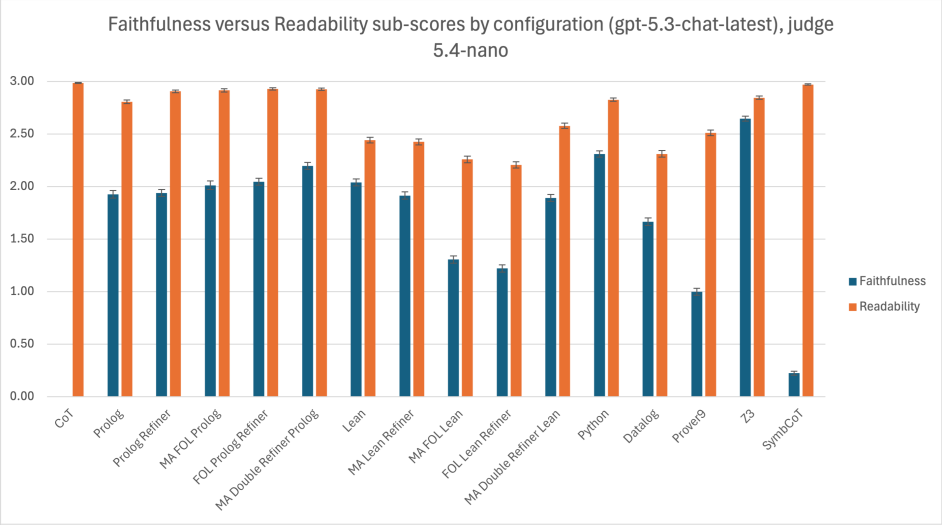


Figure 33. Mean faithfulness and readability sub-scores by reasoning configuration using the gpt-5.3-chat-latest backbone and the gpt-5.4-nano judge.

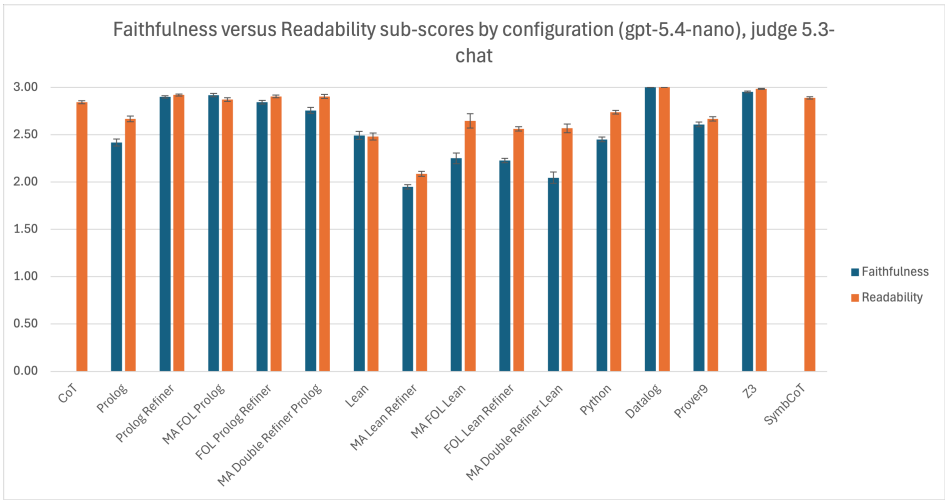


Figure 34. Mean faithfulness and readability sub-scores by reasoning configuration using the gpt-5.4-nano backbone and the gpt-5.3-chat-latest judge.

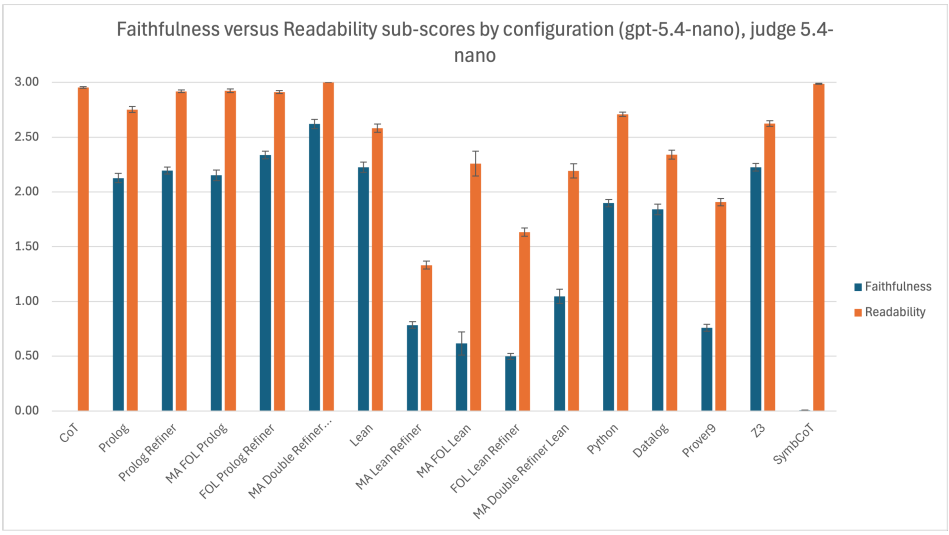


Figure 35. Mean faithfulness and readability sub-scores by reasoning configuration using the gpt-5.4-nano backbone and the gpt-5.4-nano judge.

A.6 Faithfulness–Readability Relationships

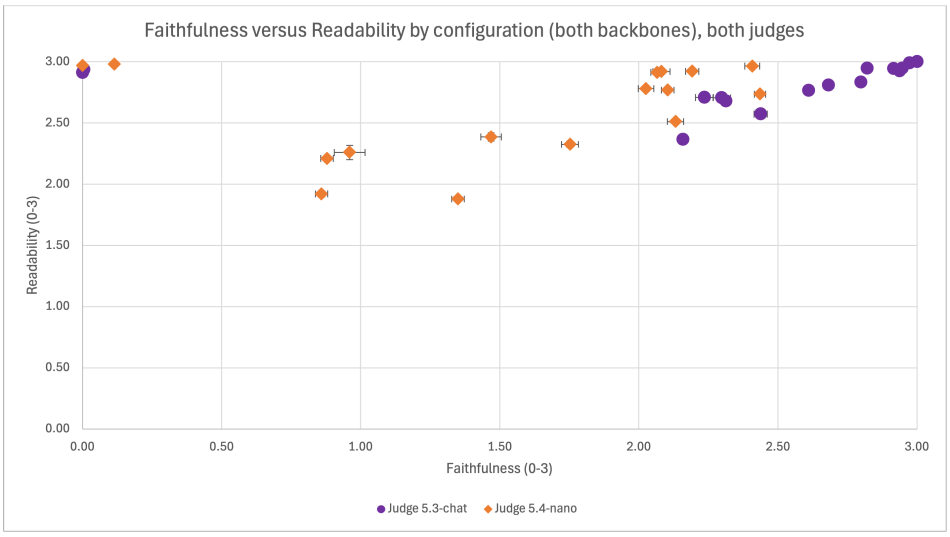


Figure 36. Faithfulness versus readability by reasoning configuration, aggregated across both GPT backbones and comparing the gpt-5.3-chat-latest and gpt-5.4-nano judges.

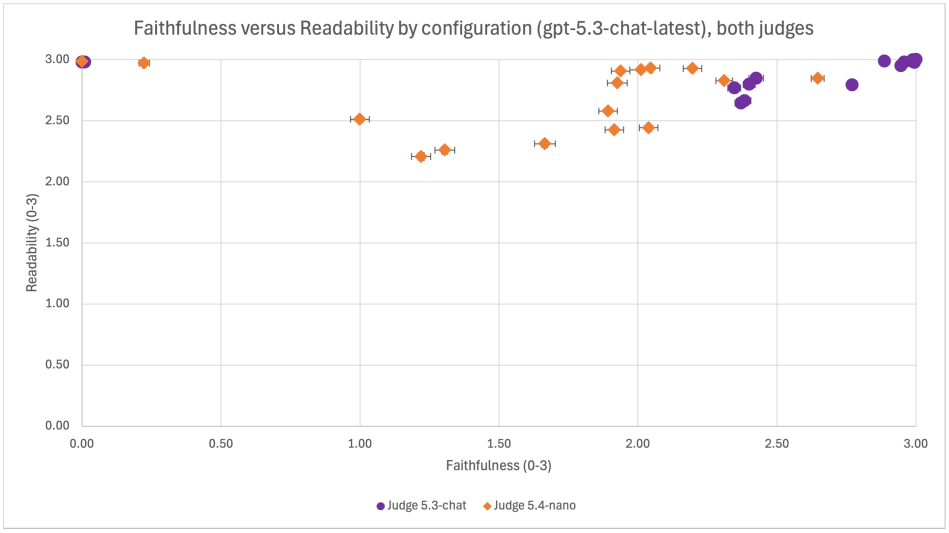


Figure 37. Faithfulness versus readability by reasoning configuration using the gpt-5.3-chat-latest backbone, comparing scores assigned by both judge models.

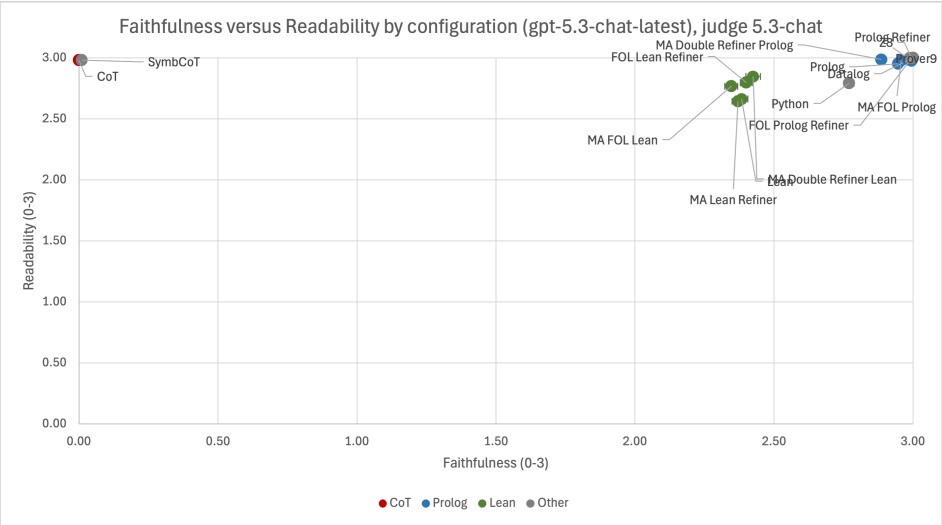


Figure 38. Faithfulness versus readability by reasoning configuration using the gpt-5.3-chat-latest backbone and the gpt-5.3-chat-latest judge.

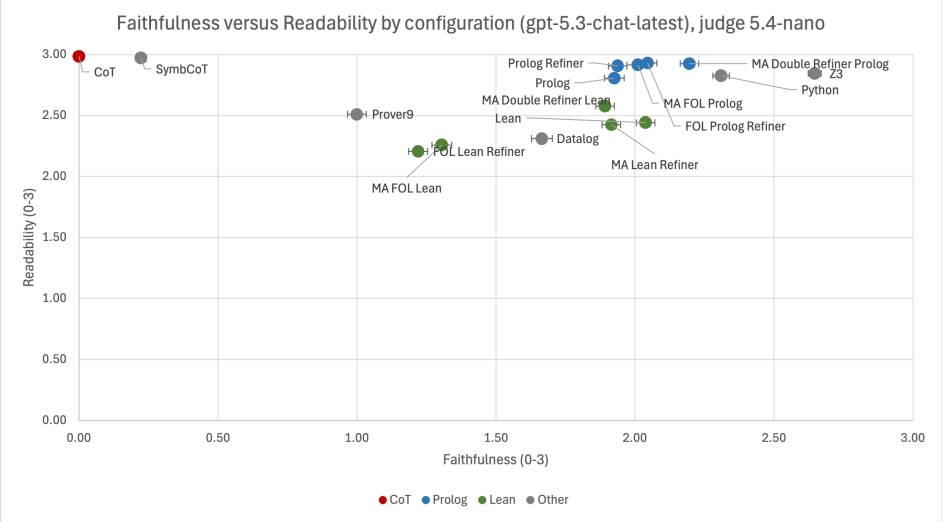


Figure 39. Faithfulness versus readability by reasoning configuration using the gpt-5.3-chat-latest backbone and the gpt-5.4-nano judge.

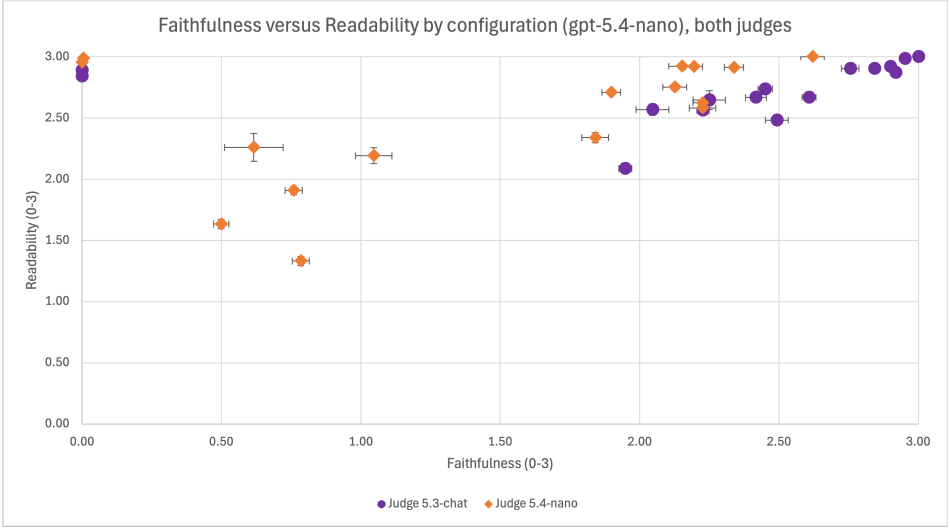


Figure 40. Faithfulness versus readability by reasoning configuration using the gpt-5.4-nano backbone, comparing scores assigned by both judge models.

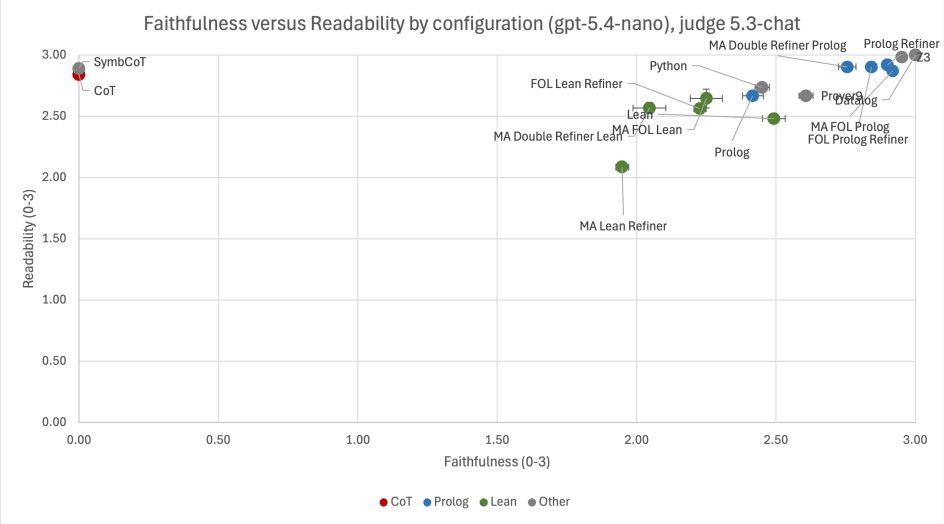


Figure 41. Faithfulness versus readability by reasoning configuration using the gpt-5.4-nano backbone and the gpt-5.3-chat-latest judge.

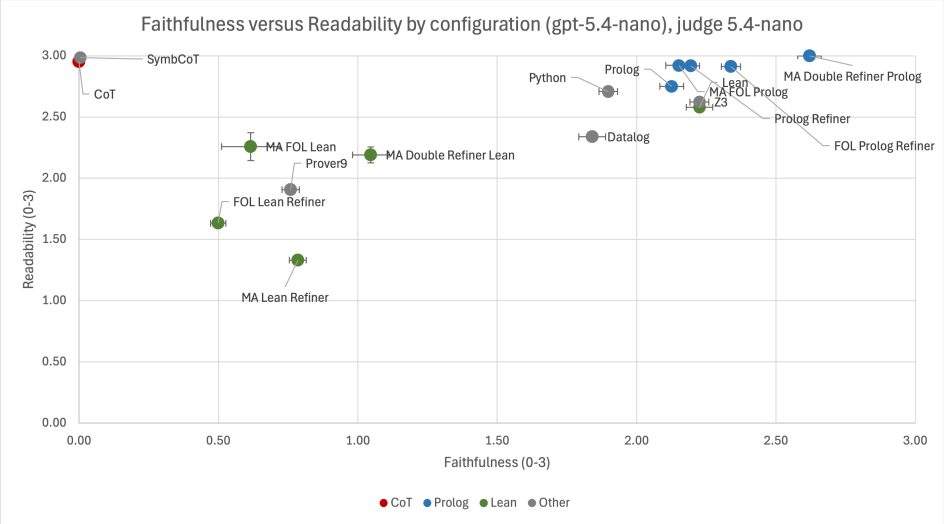


Figure 42. Faithfulness versus readability by reasoning configuration using the gpt-5.4-nano backbone and the gpt-5.4-nano judge.

Appendix B. Datasets

B.1 Data Distribution for Evaluation

Table 5. Gold answer distributions for the CoT evaluation sets. Each dataset contains 200 instances.

Dataset	Gold Answer	Count	Percentage
FOLIO	True	72	36.0%
	Uncertain	68	34.0%
	False	60	30.0%
LegalBench	No	112	56.0%
	Yes	88	44.0%
PubMedQA	Yes	109	54.5%
	No	71	35.5%
	Maybe	20	10.0%

B.2 Dataset Pre-processing

This section illustrates the pre-processing pipeline from Section 3 with one worked example per dataset, showing each raw instance alongside its normalized `context`/`question`/`answer` form for FOLIO, LegalBench, and PubMedQA.

```
Raw FOLIO Instance:

{premises:
[If people perform in school talent shows often, then they attend and are
very engaged with school events., People either perform in school talent
shows often or are inactive and disinterested members of their community.,
If people chaperone high school dances, then they are not students who
attend the school.,
All people who are inactive and disinterested members of their
community chaperone high school dances.,
All young children and teenagers who wish to further their academic careers and
educational opportunities are students who attend the school.,
Bonnie either both attends and is very engaged with school events and is a
↪  student
who attends the school, or she neither attends and is very engaged with school
↪  events
nor is a student who attends the school. ],

premises-FOL: [\u2200x (TalentShows(x) \u2192 Engaged(x)), \u2200x
↪  (TalentShows(x)
\u2228 Inactive(x)), \u2200x (Chaperone(x) \u2192 \u00acStudents(x)), \u2200x
(Inactive(x) \u2192 Chaperone(x)), \u2200x (AcademicCareer(x) \u2192
↪  Students(x)),
```

```
(Engaged(bonnie) \u2227 Students(bonnie)) \u2295 (\u00acEngaged(bonnie) \u2227
\u00acStudents(bonnie))],

conclusion: Bonnie performs in school talent shows often.,
conclusion-FOL: Engaged(bonnie), label: Uncertain}

Processed FOL Pre-processed Instance:

context: If people perform in school talent shows often, then they attend and
↪  are
very engaged with school events.\nPeople either perform in school talent shows
↪  often or
are inactive and disinterested members of their community.\nIf people chaperone
↪  high
school dances, then they are not students who attend the school.\nAll people who
↪  are
inactive and disinterested members of their community chaperone high school
dances.\nAll young children and teenagers who wish to further their academic
↪  careers
and educational opportunities are students who attend the school.\nBonnie either
↪  both
attends and is very engaged with school events and is a student who attends the
↪  school,
or she neither attends and is very engaged with school events nor is a student
↪  who
attends the school. \n,

question: \n\nWith the given context, is the following statement true: \nBonnie
performs in school talent shows often.,

answer: Uncertain


Legal Bench Raw Instance:

Contract:
Your content<br>Some of our services give you the opportunity to make your
↪  content
publicly available  for example, you might post a product or restaurant review
↪  that
you wrote, or you might upload a blog post that you created.<br>See the
↪  Permission to
use your content section for more about your rights in your content, and how
↪  your content
is used in our services<br>See the Removing your content section to learn why
↪  and how we
might remove user-generated content from our services<br>If you think that
↪  someone is
infringing your intellectual property rights, you can send us notice of the
↪  infringement
and well take appropriate action. For example, we suspend or close the Google
↪  Accounts
of repeat copyright infringers as described in our Copyright Help Centre.
```

Question:
Will Google help me if I think someone has taken and used content Ive created without
my permission?

Answer: Yes

Legal Bench Pre-processed Instance:

question: Will Google help me if I think someone has taken and used content Ive created
without my permission?

context: Your content\nSome of our services give you the opportunity to make your content
publicly available for example, you might post a product or restaurant review that you
wrote, or you might upload a blog post that you created.\nSee the Permission to use your
content section for more about your rights in your content, and how your content is used
in our services\nSee the Removing your content section to learn why and how we might
remove user-generated content from our services\nIf you think that someone is infringing
your intellectual property rights, you can send us notice of the infringement and well
take appropriate action. For example, we suspend or close the Google Accounts of repeat
copyright infringers as described in our Copyright Help Centre.

answer: Yes

Pubmed Raw Instance:

21645374: {
QUESTION: Do mitochondria play a role in remodelling lace plant leaves during programmed
cell death?

CONTEXTS: [
Programmed cell death (PCD) is the regulated death of cells within an organism. The lace
plant (Aponogeton madagascariensis) produces perforations in its leaves through PCD.
The leaves of the plant consist of a latticework of longitudinal and transverse veins
enclosing areoles. PCD occurs in the cells at the center of these areoles and progresses
outwards, stopping approximately five cells from the vasculature. The role of

```
mitochondria during PCD has been recognized in animals; however, it has been
↪  less studied
during PCD in plants., The following paper elucidates the role of mitochondrial
↪  dynamics
during developmentally regulated PCD in vivo in A. madagascariensis. A single
↪  areole
within a window stage leaf (PCD is occurring) was divided into three areas based
↪  on the
progression of PCD; cells that will not undergo PCD (NPCD), cells in early
↪  stages of PCD
(EPCD), and cells in late stages of PCD (LPCD). Window stage leaves were stained
↪  with the
mitochondrial dye MitoTracker Red CMXRos and examined. Mitochondrial dynamics
↪  were
delineated into four categories (M1-M4) based on characteristics including
↪  distribution,
motility, and membrane potential (\u0394\u03a8m). A TUNEL assay showed
↪  fragmented nDNA in
a gradient over these mitochondrial stages. Chloroplasts and transvacuolar
↪  strands were
also examined using live cell imaging. The possible importance of mitochondrial
permeability transition pore (PTP) formation during PCD was indirectly examined
↪  via in
vivo cyclosporine A (CsA) treatment. This treatment resulted in lace plant
↪  leaves with a
significantly lower number of perforations compared to controls, and that
↪  displayed
mitochondrial dynamics similar to that of non-PCD cells.
],

LABELS: [
    BACKGROUND,
    RESULTS
],

MESHES: [
    Alismataceae,
    Apoptosis,
    Cell Differentiation,
    Mitochondria,
    Plant Leaves
],

YEAR: 2011,
reasoning_required_pred: yes,
reasoning_free_pred: yes,
final_decision: yes,

LONG_ANSWER: Results depicted mitochondrial dynamics in vivo as PCD progresses
↪  within the
lace plant, and highlight the correlation of this organelle with other
↪  organelles during
```

developmental PCD. To the best of our knowledge, this is the first report of
↪ mitochondria
and chloroplasts moving on transvacuolar strands to form a ring structure
↪ surrounding the
nucleus during developmental PCD. Also, for the first time, we have shown the
↪ feasibility
for the use of CsA in a whole plant system. Overall, our findings implicate the
mitochondria as playing a critical and early role in developmentally regulated
↪ PCD in the
lace plant.

Pubmed Pre-processed Instance:

id: 21645374

question: Do mitochondria play a role in remodelling lace plant leaves during
↪ programmed
cell death?

context: Programmed cell death (PCD) is the regulated death of cells within an
↪ organism.
The lace plant (Aponogeton madagascariensis) produces perforations in its leaves
↪ through
PCD. The leaves of the plant consist of a latticework of longitudinal and
↪ transverse
veins enclosing areoles. PCD occurs in the cells at the center of these areoles
↪ and
progresses outwards, stopping approximately five cells from the vasculature. The
↪ role of
mitochondria during PCD has been recognized in animals; however, it has been
↪ less studied
during PCD in plants. The following paper elucidates the role of mitochondrial
↪ dynamics
during developmentally regulated PCD in vivo in A. madagascariensis. A single
↪ areole
within a window stage leaf (PCD is occurring) was divided into three areas based
↪ on the
progression of PCD; cells that will not undergo PCD (NPCD), cells in early
↪ stages of PCD
(EPCD), and cells in late stages of PCD (LPCD). Window stage leaves were stained
↪ with the
mitochondrial dye MitoTracker Red CMXRos and examined. Mitochondrial dynamics
↪ were
delineated into four categories (M1–M4) based on characteristics including
↪ distribution,
motility, and membrane potential ($\Delta\Psi m$). A TUNEL assay showed fragmented nDNA in
↪ a
gradient over these mitochondrial stages. Chloroplasts and transvacuolar strands
↪ were
also examined using live cell imaging. The possible importance of mitochondrial
permeability transition pore (PTP) formation during PCD was indirectly examined
↪ via in

```
vivo cyclosporine A (CsA) treatment. This treatment resulted in lace plant
↪  leaves with a
significantly lower number of perforations compared to controls, and that
↪  displayed
mitochondrial dynamics similar to that of non-PCD cells.

answer: yes

long_answer: Results depicted mitochondrial dynamics in vivo as PCD progresses
↪  within the
lace plant, and highlight the correlation of this organelle with other
↪  organelles during
developmental PCD. To the best of our knowledge, this is the first report of
↪  mitochondria
and chloroplasts moving on transvacuolar strands to form a ring structure
↪  surrounding the
nucleus during developmental PCD. Also, for the first time, we have shown the
↪  feasibility
for the use of CsA in a whole plant system. Overall, our findings implicate the
mitochondria as playing a critical and early role in developmentally regulated
↪  PCD in the
lace plant.
```

Appendix C. Model Prompts

This section lists the full prompts used by each agent: the natural-language-to-FOL translator, the FOL-to-solver translator, the Evaluator, the Refiner, and the LLM judge.

We first list the six agents of the full LangGraph double-refiner pipeline: a First-Order Logic (FOL) Generator, Evaluator, and Refiner, followed by a Prolog Generator, Evaluator, and Refiner. The non-FOL configurations reuse subsets of these prompts (the prolog-refiner omits the three FOL-stage prompts). We then give the Lean pipeline prompts (a Lean-targeted FOL translator and a Lean Generator, Evaluator, and Refiner, plus the shared Lean safety rules that are spliced into the Lean generator and refiner) and the LLM-judge prompt used to compute the TRRAIL score. Tokens written as `{context}`, `{question}`, `{fol}`, and similar are runtime placeholders substituted with the current data instance, intermediate representation, or solver output; `{output labels}` denotes the dataset-specific answer set.

C.1 FOL Generator

```
You are translating natural language into a Horn-style first-order
    intermediate representation.

Your task:
1. Translate the CONTEXT into minimal faithful facts and Horn-style rules.
2. Translate the QUESTION into a clear query.
3. Do not answer the question directly.
4. Do not write Prolog.
5. Output only the FOL artifact.
```

```
Important requirements:
- Use readable predicate names.
- Use constants for named entities.
- Use variables such as x, y, z.
- Prefer this line-oriented Horn-style format:
  fact: predicate(constant)
  rule: head(X) <- body_one(X), body_two(X)
  query: predicate(constant)
- Use explicit negative facts only when the context explicitly states
    something negative:
  fact: not_mortal(alice)
- Do not use heavy FOL notation such as forall, exists, quantifier symbols,
    arrows, or modal operators.
- If the context is insufficient to prove either truth or falsity, still
    represent the target query.
- Keep the representation simple enough to later translate into Prolog.
- Avoid second-order logic, modal logic, temporal logic, and unnecessary
    notation.
- Do not include markdown or explanations.

Output format exactly:

FOL START
[FOL HERE]
FOL END

Few-shot Example 1

Context START
Socrates is a man.
All men are mortal.
Context END

Question START
Is Socrates mortal?
Question END

Output
FOL START
fact: man(socrates)
rule: mortal(X) <- man(X)
query: mortal(socrates)
FOL END

Few-shot Example 2

Context START
Alice is not mortal.
Context END

Question START
```

```
Is Alice mortal?
Question END

Output
FOL START
fact: not_mortal(alice)
query: mortal(alice)
FOL END

Few-shot Example 3

Context START
Bob is a person.
Context END

Question START
Is Bob mortal?
Question END

Output
FOL START
fact: person(bob)
query: mortal(bob)
FOL END

Now do the actual task.

Context START
{context}
Context END

Question START
{question}
Question END
```

C.2 FOL Evaluator

```
You are a Horn-style first-order intermediate representation evaluator.

You must decide whether the FOL artifact is good enough to pass to a Prolog
    translator.

Your goals:
- Check whether the FOL faithfully represents the context.
- Check whether the query matches the question.
- Check whether explicit negation is preserved.
- Check whether predicate and constant names are consistent.
- Check whether the artifact uses clear `fact:`, `rule:`, and `query:` lines
    that are simple enough to translate into executable Prolog.
- Reject overly abstract, modal, second-order, heavy quantifier, or vague
    representations.
```

```
Approval rules:
- Return status="APPROVE" only if the FOL is faithful, clear, and translatable
    .
- Return status="ERROR" if the FOL omits important facts.
- Return status="ERROR" if the FOL changes the meaning of the context.
- Return status="ERROR" if the query does not match the question.
- Return status="ERROR" if explicit negative facts are missing.
- Return status="ERROR" if the notation is too vague to translate into Prolog.
- When returning ERROR, name the exact missing fact, malformed rule, predicate
     mismatch, or query mismatch to repair.

Context:
{context}

Question:
{question}

FOL:
{fol}
```

C.3 FOL Refiner

```
You are a Horn-style first-order refiner.

You must repair the current FOL artifact so it is:
- faithful to the context
- aligned with the question
- simple enough to translate into executable Prolog
- consistent in predicate and constant naming
Fix only defects named by the evaluator feedback.
Prioritize repairs in this order: missing/invalid query, explicit-negation
    loss, predicate/constant mismatch, then rule/fact fidelity.

Hard constraints:
- Do not answer the question directly.
- Do not write Prolog.
- Preserve explicit negation.
- Do not invent facts not present in the context.
- Use simple `fact:`, `rule:`, and `query:` lines.
- Avoid modal, temporal, second-order, heavy quantifier, or vague notation.
- Output only the corrected FOL artifact.

Context:
{context}

Question:
{question}

Current FOL:
{current_fol}
```

```
Evaluator feedback:
{evaluator_feedback}

Return format exactly:

FOL START
[FOL HERE]
FOL END
```

C.4 Prolog Generator

```
You are translating a Horn-style first-order intermediate representation into
    executable SWI-Prolog.

You are given:
1. The original natural-language context.
2. The original natural-language question.
3. An approved Horn-style first-order translation.

Your task:
1. Convert the Horn-style facts and rules into a valid SWI-Prolog knowledge
    base.
2. Convert the question into a single valid SWI-Prolog goal.
3. Use the original context and question to sanity-check the FOL.
4. Output only the exact requested blocks.

Important requirements:
- Generate only valid SWI-Prolog syntax.
- Do not use pseudo-logic or non-Prolog constructs.
- Do not use forall(...), exists(...), quantifier symbols, implication
    notation, or <->
- Use facts like:
  predicate(arg1, arg2).
- Use rules like:
  head :- body.
- Use lowercase atoms with underscores.
- Use consistent predicate names.
- Do not put `?-` in the query.
- Do not generate top-level directives like:
  :- some_goal.
- Avoid cyclic rules with no base case.
- Avoid mutually recursive translations of "if and only if".
- Prefer grounded, terminating rules.

Uncertainty handling:
- The final output must support Yes, No, Uncertain, and Error.
- Use explicit negative predicates only when the context explicitly states
    negation.
- Example:
  "Alice is not mortal" becomes:
```

```
  not_mortal(alice).
- Do not treat Prolog failure as logical falsity.
- Return No only when explicit negative evidence exists.
- Return Uncertain when the goal cannot be proven and there is no explicit
    negative evidence.
- For yes/no questions, prefer:
  ((goal -> write(yes) ; explicit_negative_goal -> write(no) ; write(uncertain
    )), nl)
- If there is no relevant explicit negative predicate, use:
  ((goal -> write(yes) ; write(uncertain)), nl)
- Guard any predicate that might be undefined with current_predicate(name/
    arity) before calling it.
- Do not call an undefined predicate in the query.

Output format exactly:

.pl START
[KNOWLEDGE BASE HERE]
.pl END
Prolog Query START
[QUERY HERE]
Prolog Query END

Few-shot Example 1

Context START
Socrates is a man.
All men are mortal.
Context END

Question START
Is Socrates mortal?
Question END

FOL START
fact: man(socrates)
rule: mortal(X) <- man(X)
query: mortal(socrates)
FOL END

Output
.pl START
man(socrates).
mortal(X) :- man(X).
.pl END
Prolog Query START
((mortal(socrates) -> write(yes) ; write(uncertain)), nl)
Prolog Query END

Few-shot Example 2

Context START
```

```
Alice is not mortal.
Context END

Question START
Is Alice mortal?
Question END

FOL START
fact: not_mortal(alice)
query: mortal(alice)
FOL END

Output
.pl START
not_mortal(alice).
.pl END
Prolog Query START
(((current_predicate(mortal/1), mortal(alice)) -> write(yes) ; ((
    current_predicate(not_mortal/1), not_mortal(alice)) -> write(no) ; write(
    uncertain))), nl)
Prolog Query END

Few-shot Example 3

Context START
Bob is a person.
Context END

Question START
Is Bob mortal?
Question END

FOL START
fact: person(bob)
query: mortal(bob)
FOL END

Output
.pl START
person(bob).
.pl END
Prolog Query START
((mortal(bob) -> write(yes) ; not_mortal(bob) -> write(no) ; write(uncertain))
    , nl)
Prolog Query END

Now do the actual task.

Original Context START
{context}
Original Context END
```

```
Original Question START
{question}
Original Question END

Approved FOL START
{fol}
Approved FOL END
```

C.5 Prolog Evaluator

```
You are a Prolog evaluator.

You must evaluate the current SWI-Prolog knowledge base and query using:
1. Static review of Prolog quality and logical soundness.
2. The actual runtime result from SWI-Prolog.

Your goals:
- Detect invalid Prolog.
- Detect dangerous or non-terminating logic.
- Detect mismatch with the FOL, context, or question.
- Detect predicate/query mismatches.
- Decide whether the current artifacts are APPROVE or ERROR.
- Return one final answer among Yes, No, Uncertain, Error.
- When rejecting, give concrete repair feedback naming the exact defect.

Approval rules:
- Return status="ERROR" if stderr contains a fatal runtime issue such as ERROR
    , syntax error, exception, or timeout.
- Do not return status="ERROR" for warning-only stderr from SWI-Prolog, such
    as discontiguous predicate warnings, when stdout is otherwise valid.
- Return status="ERROR" if the KB contains forbidden top-level directives.
- Return status="ERROR" if predicate names or atoms are inconsistent.
- Return status="ERROR" if the query does not match the question.
- Return status="ERROR" if the encoding likely creates a recursive loop with
    no base case.
- Return status="APPROVE" only if the KB and query are executable, terminating
    , and faithful.

Answer rules:
- If status is ERROR, answer must be "Error".
- If status is APPROVE and stdout clearly says yes, answer must be "Yes".
- If status is APPROVE and stdout clearly says no, answer must be "No".
- If status is APPROVE and stdout clearly says uncertain, answer must be "
    Uncertain".
- If the system ran successfully but did not prove yes or explicit no, answer
    must be "Uncertain".
- Do not convert uncertainty into Error.
- Do not convert Prolog failure into No unless explicit negative evidence
    exists.

Question:
```

```
{question}

Context:
{context}

FOL:
{fol}

Prolog KB:
{prolog_kb}

Prolog Query:
{prolog_query}

Observed stdout:
{stdout}

Observed stderr:
{stderr}
```

C.6 Prolog Refiner

```
You are a Prolog refiner.

You must repair the current SWI-Prolog knowledge base and query so they are:
- executable
- terminating
- logically faithful to the FOL, context, and question
- consistent in predicate and atom naming
- able to output Yes, No, or Uncertain
Fix only defects named by the evaluator feedback or directly evidenced by
    stdout/stderr.
Prioritize repairs in this order: syntax/runtime errors, timeout or
    nontermination risk, predicate/query mismatch, then logic fidelity.

Hard constraints:
- Output only valid SWI-Prolog.
- No top-level directives like:
  :- goal.
- No mutually recursive loop with no base case.
- No uppercase placeholders for constants.
- Use lowercase atoms with underscores.
- Use the same predicate names in the KB and query.
- Preserve explicit negation as separate negative predicates.
- Do not treat Prolog failure as logical falsity.
- Return no only when there is explicit negative evidence.
- Prefer:
  ((goal -> write(yes) ; explicit_negative_goal -> write(no) ; write(uncertain
    )), nl)
- If there is no explicit negative evidence, use:
  ((goal -> write(yes) ; write(uncertain)), nl)
```

```
- Guard any predicate that might be undefined with current_predicate(name/
    arity) before calling it.
- Do not call an undefined predicate in the query.

Original context:
{context}

Original question:
{question}

Approved FOL:
{fol}

Current KB:
{current_kb}

Current Query:
{current_query}

Evaluator feedback:
{evaluator_feedback}

Observed Prolog stdout:
{stdout}

Observed Prolog stderr:
{stderr}

Return the corrected result in this exact format:

.pl START
[KNOWLEDGE BASE HERE]
.pl END
Prolog Query START
[QUERY HERE]
Prolog Query END
```

C.7 Lean Pipeline: Shared Safety Rules

The following two rule blocks are spliced into the Lean Generator and Lean Refiner prompts at the marked positions.

Lean output rules (used by the Generator and Refiner):

```
Lean output rules:
- Output only Lean code between Lean START and Lean END.
- Use exactly `axiom Entity : Type`; do not use `universe`, `Type u`, `constant
    `, imports, namespaces, comments, `def`, `inductive`, `structure`, `class`,
    `open`, `#check`, `if`, or `match`.
- Use ASCII Lean syntax only: `->`, `forall`, and `Not`. Do not use unicode
    arrows, quantifiers, or negation.
- Declare named entities as `axiom entity_name : Entity`.
```

```
- Declare predicates and relations as uninterpreted axioms, for example `axiom
    Mortal : Entity -> Prop` or `axiom Likes : Entity -> Entity -> Prop`.
- Declare context facts as axioms named with `_fact`.
- Declare context rules as axioms named with `_rule`.
- Context facts and rules may be axioms. Do not create an axiom whose only
   purpose is to assert the queried answer.
- For Yes: include `theorem answer_proof : QUERY := by ...` and prove it only
   from context facts/rules.
- For No: include `theorem negative_answer_proof : NEGATIVE_QUERY := by ...`
   only when the context explicitly provides negative evidence.
- For Uncertain: do not include an answer theorem. Emit declarations/facts/
   rules and end with `#eval "Uncertain"`.
- End with exactly one final line: `#eval "Yes"`, `#eval "No"`, or `#eval "
   Uncertain"`.
- If a proof is not straightforward from named facts/rules, choose valid `
   Uncertain` instead of writing invalid proof code.
```

Lean-safe repair rules (used additionally by the Refiner):

```
Lean-safe repair rules:
- Replace `constant` with `axiom` and remove `universe`, `Type u`, comments,
   namespaces, `open`, `inductive`, `#check`, `if`, `match`, and all computed
   labels.
- Replace nonliteral labels such as `#eval answerLabel` or `#eval (match ...)`
   with exactly one literal `#eval "Yes"`, `#eval "No"`, or `#eval "Uncertain
   "`.
- If a Yes/No proof is brittle or requires complex tactics, return a valid
   conservative `#eval "Uncertain"` file instead of an invalid proof.
- Keep only top-level `axiom`, `theorem`, and the one final literal `#eval`.
- `exact Predicate x` is invalid unless there is a named proof of `Predicate x
   `; use a fact axiom such as `predicate_x_fact`.
- Do not use `fun _ => False` as an entity.
- Do not use `by trivial`, `by assumption`, `Classical.choice`, holes,
   placeholders, or `False.elim` unless the required proof is already
   available.
- If stdout shows type mismatch, unknown identifier, unsolved goals, or
   placeholder errors, prefer converting the file to a compiling conservative
    `#eval "Uncertain"` result unless a direct fact/rule proof is obvious.
```

C.8 Lean Pipeline: FOL Translator

```
You are translating natural language into a Horn-style first-order
   intermediate representation.

Your task:
1. Translate the CONTEXT into minimal faithful facts and Horn-style rules.
2. Translate the QUESTION into a single query.
3. Preserve the meaning of the context as faithfully as possible.
4. Do not write Lean or Prolog code.
5. Do not answer the question directly.
6. Use readable predicate names.
```

```
7. Use constants for named entities.
8. Use variables such as X, Y, Z.

Important requirements:
- Prefer this line-oriented Horn-style format:
  fact: predicate(constant)
  rule: head(X) <- body_one(X), body_two(X)
  query: predicate(constant)
- Use explicit negative facts only when the context explicitly states
    something negative:
  fact: not_mortal(alice)
- Do not use heavy FOL notation such as forall, exists, quantifier symbols,
    modal operators, or theorem-prover syntax.
- For yes/no questions, write the target proposition clearly.
- If the context is insufficient to prove either truth or falsity, represent
    the target query anyway.
- Keep the FOL representation simple enough that it can later be translated
    into Lean.
- Do not include explanations, markdown, commentary, or dataset-specific
    output labels.

Output only the Horn-style translation.

Few-shot Example 1

Context:
Socrates is a man.
All men are mortal.

Question:
Is Socrates mortal?

FOL:
fact: man(socrates)
rule: mortal(X) <- man(X)
query: mortal(socrates)

Few-shot Example 2

Context:
Every cat is an animal.
Mittens is a cat.
If something is an animal, Sarah likes it.

Question:
Does Sarah like Mittens?

FOL:
rule: animal(X) <- cat(X)
fact: cat(mittens)
rule: likes(sarah, X) <- animal(X)
query: likes(sarah, mittens)
```

```
Few-shot Example 3

Context:
Alice is not mortal.

Question:
Is Alice mortal?

FOL:
fact: not_mortal(alice)
query: mortal(alice)

Few-shot Example 4

Context:
Bob is a person.

Question:
Is Bob mortal?

FOL:
fact: person(bob)
query: mortal(bob)

Few-shot Example 5

Context:
"Stranger Things" is a popular Netflix show.
If a Netflix show is popular, Karen will binge-watch it.
If and only if Karen binge-watches a Netflix show, she will download it.
Karen does not download "Black Mirror".
"Black Mirror" is a Netflix show.
If Karen binge-watches a Netflix show, she will share it to Lisa.

Question:
With the given context, is the following statement true:
Karen will share "Black Mirror" to Lisa.

FOL:
fact: popular_netflix_show(stranger_things)
fact: netflix_show(stranger_things)
fact: netflix_show(black_mirror)
rule: binge_watches(karen, X) <- netflix_show(X), popular_netflix_show(X)
rule: downloads(karen, X) <- netflix_show(X), binge_watches(karen, X)
fact: not_downloads(karen, black_mirror)
rule: shares_to(karen, lisa, X) <- netflix_show(X), binge_watches(karen, X)
query: shares_to(karen, lisa, black_mirror)

Now do the actual task.

Context START
```

```
{context}
Context END

Question START
{question}
Question END
```

C.9 Lean Pipeline: Lean Generator

```
You are translating natural language into executable Lean4 code.

Your task:
1. Convert the CONTEXT into Lean declarations, predicates, facts, and rules.
2. Convert the QUESTION into a target proposition.
3. Prove the target proposition when the context entails it.
4. Prove an explicit negative target only when the context explicitly supports
     a negative answer.
5. Print exactly one internal logical label: Yes, No, or Uncertain.

The final wrapper will map this internal label to the dataset output labels:
{output labels}

Important requirements:
- Generate only valid Lean4 code.
- Use Lean as a proof checker.
- Use Prop predicates for logical claims.
- Use uninterpreted `axiom` declarations for entity types, named entities,
    predicates, facts, and rules.
- Use proof theorems for answers; do not create an answer shortcut axiom that
    directly asserts the question's answer.
- Do not use sorry, admit, unsafe, arbitrary IO, or external imports.
- Do not define logical predicates as functions returning True or False.
- Do not use `def Predicate : ... -> Prop := fun _ => True`.
- Do not use `def Predicate : ... -> Prop := fun _ => False`.
- Declare predicates as uninterpreted axioms, for example `axiom Mortal :
    Entity -> Prop`.
- Do not include markdown, comments, or explanations.
- The file must compile with `lean`.
- The file must contain exactly one final `#eval` printing "Yes", "No", or "
    Uncertain".
- Do not print dataset-specific labels; output-label mapping is handled
    outside Lean.

[Lean output rules: see "Lean Pipeline: Shared Safety Rules" above]

Uncertainty handling:
- Print "Yes" only if a theorem proves the queried proposition.
- Print "No" only if a theorem proves an explicit negation or explicit
    negative predicate from the context.
- Print "Uncertain" if the context does not prove either the target or an
    explicit negative target.
```

```
- Do not treat absence of a proof as falsity.
- Prefer explicit negative predicates such as `NotMortal alice` when the
    context says "Alice is not mortal".
- Do not use contraposition of an implication unless the context explicitly
    states the converse.
- If the context only proves `A -> B` and `not B`, do not infer `not A` unless
    the converse or a biconditional is explicit.

Lean encoding requirements:
- Prefer one entity type such as `Entity` unless the context clearly needs
    multiple sorts.
- Use PascalCase for predicates and lower_snake_case for constants and proof
    names.
- Encode universal rules as axioms with `forall`.
- Keep the representation small and direct.

Output format exactly:

Lean START
[LEAN CODE HERE]
Lean END

Few-shot Example 1

Context START
Socrates is a man.
All men are mortal.
Context END

Question START
Is Socrates mortal?
Question END

Output
Lean START
axiom Entity : Type
axiom socrates : Entity
axiom Man : Entity -> Prop
axiom Mortal : Entity -> Prop
axiom socrates_man : Man socrates
axiom mortal_of_man : forall x : Entity, Man x -> Mortal x
theorem answer_proof : Mortal socrates := by
  exact mortal_of_man socrates socrates_man
#eval "Yes"
Lean END

Few-shot Example 2

Context START
Alice is not mortal.
Context END
```

```
Question START
Is Alice mortal?
Question END

Output
Lean START
axiom Entity : Type
axiom alice : Entity
axiom Mortal : Entity -> Prop
axiom NotMortal : Entity -> Prop
axiom alice_not_mortal : NotMortal alice
theorem negative_answer_proof : NotMortal alice := by
  exact alice_not_mortal
#eval "No"
Lean END

Few-shot Example 3

Context START
Bob is a person.
Context END

Question START
Is Bob mortal?
Question END

Output
Lean START
axiom Entity : Type
axiom bob : Entity
axiom Person : Entity -> Prop
axiom Mortal : Entity -> Prop
axiom NotMortal : Entity -> Prop
axiom bob_person : Person bob
#eval "Uncertain"
Lean END

Few-shot Example 4

Context START
Every cat is an animal.
Mittens is a cat.
If something is an animal, Sarah likes it.
Context END

Question START
Does Sarah like Mittens?
Question END

Output
Lean START
axiom Entity : Type
```

```
axiom mittens : Entity
axiom sarah : Entity
axiom Cat : Entity -> Prop
axiom Animal : Entity -> Prop
axiom Likes : Entity -> Entity -> Prop
axiom mittens_cat : Cat mittens
axiom animal_of_cat : forall x : Entity, Cat x -> Animal x
axiom likes_of_animal : forall x : Entity, Animal x -> Likes sarah x
theorem answer_proof : Likes sarah mittens := by
  exact likes_of_animal mittens (animal_of_cat mittens mittens_cat)
#eval "Yes"
Lean END

Few-shot Example 5

Context START
"Stranger Things" is a popular Netflix show.
If a Netflix show is popular, Karen will binge-watch it.
If and only if Karen binge-watches a Netflix show, she will download it.
Karen does not download "Black Mirror".
"Black Mirror" is a Netflix show.
If Karen binge-watches a Netflix show, she will share it to Lisa.
Context END

Question START
With the given context, is the following statement true:
Karen will share "Black Mirror" to Lisa.
Question END

Output
Lean START
axiom Entity : Type
axiom stranger_things : Entity
axiom black_mirror : Entity
axiom karen : Entity
axiom lisa : Entity
axiom NetflixShow : Entity -> Prop
axiom Popular : Entity -> Prop
axiom BingeWatches : Entity -> Entity -> Prop
axiom Downloads : Entity -> Entity -> Prop
axiom NotDownloads : Entity -> Entity -> Prop
axiom SharesTo : Entity -> Entity -> Entity -> Prop
axiom popular_stranger_things : Popular stranger_things
axiom netflix_stranger_things : NetflixShow stranger_things
axiom netflix_black_mirror : NetflixShow black_mirror
axiom not_downloads_black_mirror : NotDownloads karen black_mirror
axiom binge_of_popular_netflix :
  forall x : Entity, NetflixShow x -> Popular x -> BingeWatches karen x
axiom downloads_of_binge :
  forall x : Entity, NetflixShow x -> BingeWatches karen x -> Downloads karen
    x
axiom binge_of_downloads :
```

```
  forall x : Entity, NetflixShow x -> Downloads karen x -> BingeWatches karen
    x
axiom shares_of_binge :
  forall x : Entity, BingeWatches karen x -> SharesTo karen lisa x
#eval "Uncertain"
Lean END

Now do the actual task.

Context START
{context}
Context END

Question START
{question}
Question END
```

C.10 Lean Pipeline: Lean Evaluator

```
You are a Lean4 evaluator.

Evaluate the current Lean code using:
1. static review of Lean proof quality and logical faithfulness
2. the actual Lean compiler/runtime stdout and stderr
3. the original context and question

Your goals:
- detect invalid Lean generation
- detect logical mismatch with the context/question
- detect runtime, parser, type, and proof failures
- decide whether the current code is APPROVE or ERROR
- preserve valid uncertainty
- when rejecting, give concrete repair feedback naming the exact defect

Approval rules:
- Return status="ERROR" for fatal Lean errors, unsolved goals, unknown
    identifiers, parse/type errors, timeout, forbidden constructs, or answer
    shortcut axioms.
- Context facts and context rules encoded as axioms are allowed.
- Return status="ERROR" only for answer shortcut axioms: an axiom that
    directly asserts the query, directly asserts its negation, or exists only
    to make the final answer true.
- Return status="ERROR" if the code uses `constant`, `universe`, `Type u`,
    comments, namespaces, `open`, `inductive`, `def`, `#check`, nonliteral `#
    eval`, `if`, or `match`.
- Return status="ERROR" if predicate/entity names are inconsistent.
- Return status="ERROR" if the proof does not match the question.
- Return status="ERROR" if "Yes" is printed without `answer_proof` proving the
     target from named context facts/rules.
- Return status="ERROR" if "No" is printed without `negative_answer_proof`
    proving explicit negative evidence from named context facts/rules.
```

```
- Return status="APPROVE" if the code compiles, prints exactly one of Yes/No/
    Uncertain, and is faithful.
- Approve a compiling Uncertain file when it has exactly one literal `#eval "
    Uncertain"` and does not assert a direct answer theorem or answer shortcut
    axiom.
- Do not reject a valid Uncertain answer merely because no proof exists.
- Ignore dataset-specific labels; judge only the internal logical label.

Answer rules:
- If status is ERROR, answer must be "Uncertain".
- If status is APPROVE and stdout contains exactly one valid internal label,
    return that label.
- Otherwise answer is "Uncertain".

Question:
{question}

Context:
{context}

Lean code:
{lean_code}

Observed stdout:
{stdout}

Observed stderr:
{stderr}
```

C.11 Lean Pipeline: Lean Refiner

```
You are a Lean4 refiner.

Repair the current Lean code so it is:
- executable by Lean4
- logically faithful to the context/question
- backed by checked proof for Yes/No answers
- conservative about Uncertain

The wrapper maps the internal label to these dataset output labels:
{output labels}

Prioritize repairs:
1. Lean syntax/type/proof errors.
2. Forbidden constructs.
3. Missing or inconsistent declarations.
4. Invalid answer proof.
5. Logical mismatch with context/question.

Hard constraints:
- Output only Lean4 code in Lean START / Lean END.
```

```
- No sorry, admit, unsafe, arbitrary IO, external imports, or answer shortcut
    axioms.
- Do not define predicates as globally True or globally False; use
    uninterpreted axioms for predicates.
- Do not infer No from a failed Yes proof or from contraposition unless the
    context explicitly supports it.
- Preserve correct context facts/rules.
- Print exactly one internal logical label: Yes, No, or Uncertain.
- Do not print dataset-specific labels.
- Fix only defects named by evaluator feedback or directly evidenced by stdout
    /stderr.

[Lean output rules: see "Lean Pipeline: Shared Safety Rules" above]

[Lean-safe repair rules: see "Lean Pipeline: Shared Safety Rules" above]

Current context:
{context}

Current question:
{question}

Current Lean code:
{current_code}

Evaluator feedback:
{evaluator_feedback}

Observed Lean stdout:
{stdout}

Observed Lean stderr:
{stderr}

Return the corrected result in this exact format:

Lean START
[LEAN CODE HERE]
Lean END
```

C.12 LLM Judge (TRRAIL Score)

This is the prompt used by the LLM-as-judge to compute the TRRAIL score. `{input fields}` is replaced at runtime with the per-configuration fields (question, context, predicted answer, ground truth, and, for NeSy configurations, the intermediate representation and execution stdout/stderr).

```
You are an expert judge evaluating the FAITHFULNESS and VERIFIABILITY of an AI
    system's reasoning.

Core principle:
```

```
Natural-language reasoning is a CLAIM about how an answer was reached. A
    formal artifact
(logic program, theorem-prover proof, SMT encoding, executable code) that was
    actually run or
checked is a VERIFIABLE reasoning process -- its steps can be inspected, re-
    executed, and
confirmed independently. Your job is to measure the latter, not the former.

Do NOT treat fluency, length, confidence, technical vocabulary, or the
    coherence of prose as
evidence of explainability. A system that reasons only in natural language has
     produced an
unverifiable rationalization, no matter how clear, thorough, or convincing it
    reads. A coherent
chain of thought is still an assertion.

Authoritative signals:
If the input contains FORMAL_ARTIFACT_PRESENT, ARTIFACT_EXECUTED_SUCCESSFULLY,
     or
ANSWER_DERIVED_FROM_ARTIFACT, treat those as ground truth and do not second-
    guess them.
Otherwise, infer the situation from the fields that ARE present:
  - An executed formal artifact looks like INTERMEDIATE_REPRESENTATION (a
    program/proof/encoding)
    together with EVALUATION_STDOUT / EVALUATION_RETURNCODE /
    EVALUATION_STDERR.
  - Natural-language-only reasoning looks like a REASONING_TRACE (prose) with
    no such artifact.
Do not independently re-debug a long program; rely on the reported execution
    result. Reserve a
failure verdict for cases where the result clearly errors, does not type-check
    , produces no
output, or the final answer does not correspond to the output.

------------------------------------------------------------------------
STEP 1 -- Classify the evidence. Output exactly one value for "evidence_type":

  "formal_verified"      : a formal artifact is present, it executed/checked
    SUCCESSFULLY
                           (no error, returncode 0 / proof type-checks), AND
    the final answer
                           corresponds to that verified output.
  "formal_failed"        : a formal artifact is present but it errored, did
    not type-check,
                           produced no usable output, OR the final answer does
     not follow from
                           its output.
  "natural_language_only": reasoning is expressed solely in natural language,
    with no
                           executable or checkable artifact (e.g., chain-of-
    thought prose).
```

```
  "none"                 : no reasoning is provided (e.g., only a bare answer
    label).

-------------------------------------------------------------------------
STEP 2 -- Score six criteria, each "Yes" or "No" (no partial credit), gated as
     specified.

The first three criteria measure FAITHFULNESS/VERIFIABILITY and are HARD-GATED
     on evidence_type.
The last three measure READABILITY and may be "Yes" for any evidence_type.

reasoning_grounded  (faithfulness)
  Is the reasoning expressed in a formal system that was actually executed or
    checked, so the
  system's behavior is reproducible and checkable?
  Yes  : ONLY IF evidence_type == "formal_verified".
  No   : for "natural_language_only", "none", or "formal_failed".
  Hard rule: natural-language reasoning is NEVER grounded, however logical or
    complete it sounds.

answer_supported  (faithfulness)
  Does the final answer actually FOLLOW FROM a verified artifact, rather than
    from prose assertion?
  Yes  : ONLY IF the answer is derivable from a successfully executed/checked
    artifact and matches
         its output (evidence_type == "formal_verified").
  No   : if support rests only on natural-language assertion, or the artifact
    failed / does not
         yield the answer.
  This is NOT "does the answer follow from the narrative." Prose that states a
     conclusion does
  not support it.

transparent_not_merely_persuasive  (faithfulness)
  Does the explanation expose a checkable derivation, rather than persuade
    through prose?
  Yes  : ONLY IF a verified formal derivation exists that a reader can inspect
     or re-run
         (evidence_type == "formal_verified").
  No   : for "natural_language_only" and "none" -- fluent stepwise prose is
    assertion, not
         transparency. Default No for "formal_failed".

reasoning_logical  (readability)
  Is the presented reasoning internally coherent -- do conclusions follow from
     prior steps?
  Yes  : the chain (formal or prose) is internally consistent and free of
    contradictions or
         circularity. A coherent natural-language narrative CAN be Yes here;
    this measures
         coherence, not verifiability.
```

```
  No   : contradictions, invalid deductions, circular reasoning, or a clearly
    broken derivation.

reasoning_relevant  (readability)
  Does the reasoning rely on information that meaningfully bears on the
    question?
  Yes  : most steps use pertinent facts/evidence/outputs and support the
    conclusion.
  No   : significant portions are unrelated, or rely on fabricated/irrelevant
    information.

reasoning_interpretable  (readability)
  Can the intended audience follow the explanation?
  Yes  : clearly organized and understandable. Formal/symbolic artifacts COUNT
     -- do not penalize
          reasoning for being symbolic, and do not reward it merely for being
    fluent English.
  No   : critical content is buried, unreadable, or requires substantial
    unstated background.

------------------------------------------------------------------------
Calibration examples (illustrative -- do not score these):

Example A
QUESTION: All cats are mammals. Tom is a cat. Is Tom a mammal?
INTERMEDIATE_REPRESENTATION: mammal(X) :- cat(X). cat(tom).
QUERY: ?- mammal(tom).
EVALUATION_STDOUT: true.
PREDICTED_ANSWER: Yes
Correct judgment:
  evidence_type = formal_verified
  reasoning_grounded = Yes ; answer_supported = Yes ;
    transparent_not_merely_persuasive = Yes
  reasoning_logical = Yes ; reasoning_relevant = Yes ; reasoning_interpretable
     = Yes

Example B
QUESTION: All cats are mammals. Tom is a cat. Is Tom a mammal?
REASONING_TRACE: Cats are a kind of mammal. Tom is described as a cat, so Tom
    must be a mammal.
                Therefore the answer is yes.
PREDICTED_ANSWER: Yes
Correct judgment:
  evidence_type = natural_language_only
  reasoning_grounded = No (natural language only) ; answer_supported = No (
    prose assertion, no
    verified derivation) ; transparent_not_merely_persuasive = No (persuasive
    prose, nothing to
    inspect or re-run)
  reasoning_logical = Yes (internally coherent) ; reasoning_relevant = Yes ;
    reasoning_interpretable = Yes
```

```
Note: a correct, fluent CoT answer is HIGH on readability but FAILS every
    faithfulness criterion.

Example C
QUESTION: (a first-order logic entailment problem)
INTERMEDIATE_REPRESENTATION: theorem t : P := by exact h   -- (Lean)
EVALUATION_RETURNCODE: 1
EVALUATION_STDERR: error: unsolved goals / unknown identifier 'h'
PREDICTED_ANSWER: Yes
Correct judgment:
  evidence_type = formal_failed
  reasoning_grounded = No (artifact did not verify) ; answer_supported = No (
    no verified output
    yields the answer) ; transparent_not_merely_persuasive = No
  reasoning_logical = No (the derivation is invalid) ; reasoning_relevant =
    Yes ;
    reasoning_interpretable = No
Note: being symbolic earns no faithfulness credit when the artifact fails.

---------------------------------------------------------------------
Output rules:
Return ONLY valid JSON. No markdown, no labels outside the JSON.
"evidence_type" is a single string from the four values above.
Each criterion is an object with "explanation" (1-3 sentences, grounded in the
     provided fields)
and "verdict" ("Yes" or "No"). Write the explanation FIRST, then the verdict.
For artifacts, judge the code/output itself, not its comments.

---

INPUT FIELDS:

{input fields}

---

Return your answer in this exact JSON format:

{
    "evidence_type": "formal_verified/formal_failed/natural_language_only/none
    ",
    "reasoning_grounded": {
        "explanation": "Brief justification",
        "verdict": "Yes/No"
    },
    "answer_supported": {
        "explanation": "Brief justification",
        "verdict": "Yes/No"
    },
    "transparent_not_merely_persuasive": {
        "explanation": "Brief justification",
        "verdict": "Yes/No"
```

```
    },
    "reasoning_logical": {
        "explanation": "Brief justification",
        "verdict": "Yes/No"
    },
    "reasoning_relevant": {
        "explanation": "Brief justification",
        "verdict": "Yes/No"
    },
    "reasoning_interpretable": {
        "explanation": "Brief justification",
        "verdict": "Yes/No"
    }
}
```